

Python 程序设计基础

完整复习资料（含知识点、题库、模拟试 卷与速背手册）

子冉一人 整理

2026 年 7 月 12 日

目录

第一部分 章节知识点（第 1–9 章）	3
第一章 第 1 章 Python 语言概述	4
1.1 知识要点	4
1.2 经典选择题	4
1.3 易错概念辨析题	6
1.4 简答题	6
1.5 课后作业	7
第二章 第 2 章运算符、表达式与内置对象	9
2.1 知识要点	9
2.2 经典选择题	10
2.3 易错概念辨析题	14
2.4 简答题	17
2.5 编程题	18
2.6 课后作业	19
第三章 第 3 章 Python 可迭代对象（序列结构）	21
3.1 知识要点	21
3.2 经典选择题	22
3.3 易错概念辨析题	26
3.4 简答题	29
3.5 编程题	30
3.6 课后作业	31
第四章 第 4 章程序控制结构	36
4.1 知识要点	36
4.2 经典选择题	37

4.3	易错概念辨析题	39
4.4	简答题	39
4.5	编程题	40
4.6	课后作业	41
第五章	第 5 章函数	46
5.1	知识要点	46
5.2	经典选择题	47
5.3	易错概念辨析题	50
5.4	简答题	52
5.5	编程题	54
5.6	课后作业	55
第六章	第 6 章面向对象程序设计	59
6.1	知识要点	59
6.2	经典选择题	60
6.3	易错概念辨析题	61
6.4	简答题	63
6.5	编程题	64
第七章	第 7 章字符串	65
7.1	知识要点	65
7.2	经典选择题	66
7.3	易错概念辨析题	67
7.4	简答题	68
7.5	编程题	69
第八章	第 8 章文件操作	70
8.1	知识要点	70
8.2	经典选择题	71
8.3	易错概念辨析题	72
8.4	课后作业	72

第九章 第 9 章异常处理	75
9.1 知识要点	75
9.2 异常处理错题精选	76
 第二部分 第 6 章面向对象程序设计详解	 78
第十章 类的定义与使用	79
10.1 定义类的语法	79
10.2 示例	79
 第十一章 数据成员与成员方法	 80
11.1 私有成员与公有成员（必考）	80
11.2 数据成员：实例属性 vs 类属性	81
11.3 成员方法、类方法、静态方法、抽象方法	81
11.4 属性（Property）	82
11.5 动态性与混入机制（Mixin）	83
 第十二章 继承与多态	 84
12.1 继承	84
12.2 多态	85
 第十三章 特殊方法与运算符重载	 86
 第十四章 本章重点案例赏析	 88
14.1 自定义队列类	88
14.2 自定义栈类	89
 第十五章 考试要点浓缩	 90
 第十六章 本章作业题	 91

第三部分 考前速背手册 93**第四部分 模拟试卷 101****第十七章 模拟试卷一 102**

17.1 选择题（每题 2 分，共 40 分）	102
17.2 判断题（每题 2 分，共 20 分）	104
17.3 简答题（每题 10 分，共 20 分）	104
17.4 编程题（每题 15 分，共 30 分）	104
17.5 参考答案	106

第十八章 模拟试卷二 107

18.1 选择题（每题 2 分，共 40 分）	107
18.2 判断题（每题 2 分，共 20 分）	109
18.3 简答题（每题 10 分，共 20 分）	109
18.4 编程题（每题 15 分，共 30 分）	109
18.5 参考答案	110

附录 A 错题集精选 111

A.1 变量引用与对象绑定	111
A.2 默认值参数陷阱	113
A.3 变量作用域	113
A.4 推导式与运算符	114
A.5 字典与集合	115
A.6 循环控制	116
A.7 函数调用综合	116

第一部分

章节知识点（第 1–9 章）

第 1 章 第 1 章 Python 语言概述

知识要点

- Python 特性：跨平台、开源、免费、解释型高级动态编程语言（了解）
- 编程规范与代码优化：缩进（4 空格）、空格与空行、注释（# 和三引号）、不同数据类型对效率的影响（了解）
- 三种导入方法及其区别：
 1. `import 模块名 [as 别名]` ——使用时加模块名前缀
 2. `from 模块名 import 对象名 [as 别名]` ——直接使用对象名
 3. `from 模块名 import *` ——导入全部对象（不推荐，命名空间混乱）
- `__name__` 属性的作用：识别程序的运行方式
 - 直接运行：`__name__ == '__main__'`
 - 作为模块导入：`__name__` 为模块文件名
- `if __name__ == '__main__':`：常用于模块自测

删除内容：课件中有关计算机操作的内容（如安装配置、IDE 使用等）不考。

经典选择题

选择题 1-1

下面属于 Python 语言特点的是（ ）。(多选)

- A. 开源
- B. 免费
- C. 跨平台
- D. 解释执行

参考答案与解析

答案：ABCD

解析：Python 语言最主要的四个特点：跨平台（可在 Windows、Linux、macOS 上运行）、开源（源代码公开）、免费（无需付费）、解释型高级动态编程语言。

选择题 1-2

下面导入标准库对象的语句中正确是（ ）。（多选）

- A. `from math import sin`
- B. `from random import random`
- C. `from math import *`
- D. `import *`

参考答案与解析

答案：ABC

解析：A 和 B 是从模块导入指定对象，C 是导入模块所有对象，D 错误——`import *` 缺少模块名，语法不合法。

选择题 1-3

`__name__` 属性的作用是（ ）。

- A. 查看模块名称
- B. 识别程序的运行方式（直接运行还是被导入）
- C. 获取函数名称
- D. 定义全局变量

参考答案与解析

答案：B

解析：`__name__` 是 Python 内置属性，直接运行时值为 `'__main__'`，被导入时值为模块文件名，用于识别程序运行方式。

易错概念辨析题

辨析题 1-1

判断对错：在编写 Python 程序时，对代码进行缩进只是为了好看，不缩进也不影响程序的正常执行。

参考答案与解析

答案：× 错误。

解析：Python 中缩进是代码分组的唯一方式，是语法层面的硬性要求。缩进不对会导致语法错误或逻辑错误，程序无法正常运行。

辨析题 1-2

判断对错：使用 `from math import *` 这种导入方式是推荐的 Python 编程写法。

参考答案与解析

答案：× 错误。

解析：不推荐。这种方式会污染命名空间，如果多个模块中有同名对象，会导致混乱，不利于代码阅读和维护。推荐使用 `from math import sin` 或 `import math`。

辨析题 1-3

判断对错：Python 程序文件的扩展名只有 `.py` 一种。

参考答案与解析

答案：× 错误。

解析：Python 程序文件主要有 `.py`（普通程序文件）和 `.pyw`（用于 GUI 程序，运行时不会弹出控制台窗口）两种扩展名。

简答题

简答 1-1

简述 Python 三种导入方法的区别。

参考答案与解析

答案：

1. `import` 模块名：导入整个模块，使用时需加模块名前缀，如 `math.sin(3)`，避免命名冲突。
2. `from` 模块名 `import` 对象名：仅导入指定对象，直接使用对象名，如 `sin(3)`，减少输入量。
3. `from` 模块名 `import *`：导入模块中所有对象，不推荐使用，会造成命名空间混乱。

简答 1-2

`__name__` 属性的作用是什么？写出使用它进行模块自测的典型代码。

参考答案与解析

答案： `__name__` 用于识别程序是直接运行还是被导入。

典型代码：

```
if __name__ == '__main__':  
    # 当程序直接运行时执行此处的测试代码  
    main()
```

课后作业

作业 1.23

Python 程序的 `__name__` 属性的作用是什么？

参考答案与解析

答案：__name__ 是 Python 的内置属性，核心作用是标识模块的执行状态：

1. 当模块作为主程序直接运行时，__name__ 的值为 __main__；
2. 当模块被导入到其他程序中使用，__name__ 的值为模块的文件名。

利用该属性可实现模块自测：

```
if __name__ == '__main__':  
    # 测试代码  
    pass
```

第 2 章 第 2 章运算符、表达式与内置对象

知识要点

- **变量命名规范**：以字母、汉字或下画线开头，不能包含空格或标点符号，不能使用关键字作为变量名
- **Python 采用基于值的内存管理模式**：变量不直接存储值，而是存储值的引用
- **常用运算符（必背）**：

类别	运算符	说明/注意点
算术	+ - * / // % **	/为真除法（浮点数），//为整除（向下取整），** 为幂
比较	== != > < >= <=	支持连用如 1 < x < 10
成员	in, not in	判断元素是否在序列中
逻辑	and, or, not	短路求值，返回最后一个被计算的表达式的值

- **逻辑运算符的核心规则**：
 - **and**：若左边为假则直接返回左边值（不计算右边），否则返回右边值
 - **or**：若左边为真则直接返回左边值（不计算右边），否则返回右边值
 - 例：3 and 5 返回 5；3 or 5 返回 3
- **常用内置函数（必背）**：

函数	调用示例	作用	注意点
<code>int()</code>	<code>int('123')</code>	转整数	也可 <code>int('11111', 2)</code> 按进制转
<code>float()</code>	<code>float('3.14')</code>	转浮点数	
<code>str()</code>	<code>str(123)</code>	转字符串	
<code>list()</code>	<code>list('abc')</code>	转列表	参数为可迭代对象
<code>tuple()</code>	<code>tuple([1,2])</code>	转元组	参数为可迭代对象
<code>dict()</code>	<code>dict([('a',1)])</code>	转字典	参数为键值对序列
<code>set()</code>	<code>set([1,2,3])</code>	转集合	自动去重
<code>len()</code>	<code>len([1,2,3])</code>	返回序列长度	支持所有序列
<code>max()/min()</code>	<code>max([1,3,2])</code>	最值	字符串按字典序
<code>sum()</code>	<code>sum([1,2,3])</code>	求和	元素必须为数值
<code>range()</code>	<code>range(5)</code>	生成整数范围	<code>start/stop/step</code> ，左闭右开
<code>enumerate()</code>	<code>enumerate(['a','b'])</code>	返回 (索引, 值) 对	返回可迭代对象，常与 <code>for</code> 搭配
<code>map()</code>	<code>map(int, ['1','2'])</code>	对每个元素用函数	返回 <code>map</code> 对象（迭代器）
<code>zip()</code>	<code>zip([1,2], ['a','b'])</code>	打包多序列	按最短对齐，返回迭代器
<code>sorted()</code>	<code>sorted([3,1,2])</code>	排序返回新列表	不改变原序列
<code>reversed()</code>	<code>reversed([1,2,3])</code>	反向迭代器	返回迭代器对象
<code>eval()</code>	<code>eval('1+2')</code>	执行表达式字符串	注意安全
<code>input()</code>	<code>input('提示:')</code>	键盘输入	返回字符串
<code>print()</code>	<code>print('hello')</code>	输出	<code>sep</code> 、 <code>end</code> 参数
<code>bool()</code>	<code>bool(0)</code>	判断布尔值	0/空/None 为 False

删除内容：位运算符（& | ^ ~ << >>）、集合运算符、矩阵运算不考。

经典选择题

选择题 2-1

表达式 `int('11111', 2)` 的值为（ ）。

- A. 11111
- B. 31
- C. 17
- D. 15

参考答案与解析

答案： B

解析： `int('11111', 2)` 以二进制解析字符串'11111'，即 $1 + 2 + 4 + 8 + 16 = 31$ 。

选择题 2-2

表达式 `3 and 5` 的值为 ()。

- A. 3
- B. 5
- C. 8
- D. True

参考答案与解析

答案： B

解析： `and` 返回最后一个被计算的表达式的值。因为 3 为真（非零），需要继续计算 5 才能确定结果，返回 5。

选择题 2-3

表达式 `3 or 5` 的值为 ()。

- A. 3
- B. 5
- C. 8
- D. True

参考答案与解析

答案： A

解析： `or` 返回最后一个被计算的表达式的值。因为 3 为真（非零），无需继续计算 5，直接返回 3。

选择题 2-4

已知 $x = [1, 2]$ 和 $y = [3, 4]$ ，那么 $x + y$ 的结果是（ ）。

- A. 3
- B. 7
- C. [4, 6]
- D. [1, 2, 3, 4]

参考答案与解析

答案：D

解析：运算符 $+$ 用于列表时执行连接操作，将两个列表拼接为一个新列表，结果为 `[1, 2, 3, 4]`。注意 $+$ 对数字做加法、对字符串/列表做连接（多重含义）。

选择题 2-5

下面不是合法变量名的有（ ）。（多选）

- A. age
- B. 3_name
- C. for
- D. _name

参考答案与解析

答案：BC

解析：变量名不能以数字开头（B），也不能使用 Python 关键字（C）。`_name` 以下画线开头是合法的（D），age 是合法的（A）。

选择题 2-6

表达式 `eval('3*2'+ '22')` 的值为 ()。

- A. 28
- B. 666
- C. '3*222'
- D. 622

参考答案与解析

答案： B

解析： 先字符串拼接得 '3*222'，再 `eval` 计算得 $3 \times 222 = 666$ 。

选择题 2-7

表达式 `max(['121', '34'])` 的值为 ()。

- A. '121'
- B. '34'
- C. 121
- D. 34

参考答案与解析

答案： B

解析： 字符串比较按字典序（逐位比较 ASCII 码）。'3' 的 ASCII 码 (51) > '1' 的 ASCII 码 (49)，因此 '34' > '121'。

选择题 2-8

表达式 `13 // 4` 和 `-13 // 4` 的值分别为 ()。

- A. 3 和 -3
- B. 3 和 -4
- C. 4 和 -3
- D. 4 和 -4

参考答案与解析

答案： B

解析：//为整除（向下取整）。 $13/4 = 3.25$ 向下取整得 3； $-13/4 = -3.25$ 向下取整得 -4。

选择题 2-9

关键字 `del` 可以用来（ ）。（多选）

- A. 删除列表中的一个元素
- B. 删除整个列表
- C. 删除字典中的一个键值对
- D. 删除一个变量

参考答案与解析

答案： ABCD

解析：`del` 可以删除列表元素、整个列表/变量、字典的键值对等，是非常通用的删除命令。

选择题 2-10

Python 关键字中用来判断两个对象的引用是否相同的是（ ）。

- A. `==`
- B. `=`
- C. `is`
- D. `in`

参考答案与解析

答案： C

解析：`is` 判断两个变量是否指向同一个内存地址（引用相同），而 `==` 只判断值是否相等。

易错概念辨析题

辨析题 2-1

判断对错：语句 `x = input(3)` 的作用是自动输入 3 并赋值给变量 `x`。

参考答案与解析

答案：× 错误。

解析：`input()` 的参数是提示文字，而不是自动输入的值。该语句会在屏幕上显示”3” 作为提示信息，需要用户手动输入内容后才能赋值给 `x`。

辨析题 2-2

判断对错：/ 和 // 运算符含义相同，都表示除法。

参考答案与解析

答案：× 错误。

解析：/ 是真除法，结果始终为浮点数（如 `4/2` 得 `2.0`）。// 是整除（向下取整），结果为整数（如 `4//2` 得 `2`，`-3//2` 得 `-2`）。两者完全不同。

辨析题 2-3

判断对错：加法运算符可以连接两个字典得到新字典。

参考答案与解析

答案：× 错误。

解析：加法运算符 `+` 支持数字加法、字符串连接、列表连接和元组连接，但不支持字典和集合的连接。

辨析题 2-4

判断对错：`9999**9999` 这样的表达式在 Python 中无法运行，因为计算结果太大了。

参考答案与解析

答案：× 错误。

解析：Python 的 `int` 类型整数大小没有限制（仅受内存制约），`9999**9999` 可以正常计算。Python 内置支持大整数运算是其重要特点。

辨析题 2-5

判断对错：已知 `x = '[1,2,3,4]'`，那么表达式 `list(x)` 的值为 `[1,2,3,4]`。

参考答案与解析

答案：× 错误。

解析：`list(x)` 将字符串 `'[1,2,3,4]'` 逐个字符转换为列表，结果为 `['[', '1', ',', '2', ',', '3', ',', '4', ']']`。要将字符串形式的列表转为真正的列表，应使用 `eval(x)` 得 `[1,2,3,4]`。

辨析题 2-6

判断对错：`bool(range(8, 5))` 的值为 `True`。

参考答案与解析

答案：× 错误。

解析：虽然 `range(8, 5)` 是一个有效的 `range` 对象（空范围），但 `bool(range(8,5))` 实际上返回 `False`——因为 Python 3.12 及之前版本中，空 `range` 对象的布尔值为 `False`。

辨析题 2-7

判断对错：Python 属于强类型编程语言。

参考答案与解析

答案：✓ 正确。

解析：Python 是强类型语言，不同类型之间不会自动转换。例如 `'a'+1` 会抛出 `TypeError` 异常，不会隐式转换。

辨析题 2-8

判断对错：在 Python 中可以使用 `for`、`while`、`if` 作为变量名。

参考答案与解析

答案：× 错误。

解析：`for`、`while`、`if` 是 Python 关键字，不能作为变量名使用。

辨析题 2-9

判断对错：`len(zip([1,2,3], 'abcdefg'))` 的值为 3。

参考答案与解析

答案：✓ 正确。

解析：zip 按最短可迭代对象配对。列表长度为 3，字符串长度为 7，按短的来，所以结果只有 3 对，长度为 3。

辨析题 2-10

判断对错：表达式 $3+4j$ 不是合法的 Python 表达式。

参考答案与解析

答案：× 错误。

解析： $3+4j$ 是合法的 Python 复数表达式，3 为实部， $4j$ 为虚部。Python 内置支持复数类型。

简答题

简答 2-1

简述 Python 运算符 / 和 // 的区别，并举例说明。

参考答案与解析

答案：

- / 是真除法，结果始终为浮点数。如 $4/2$ 得 2.0。
- // 是整除，向下取整，结果类型与操作数相关。如 $4//2$ 得 2， $-3//2$ 得 -2（因为 -1.5 向下取整是 -2）。

简答 2-2

简述 Python 运算符多重含义的表现，举至少三个例子。

参考答案与解析

答案：

1. +：数字加法（ $3+5=8$ ）/ 字符串连接（`'a'+'b'='ab'`）/ 列表连接（`[1]+[2]=[1,2]`）
2. *：数字乘法（ $3*5=15$ ）/ 序列重复（`'ab'*3='ababab'`）/ 列表重复（`[1]*3=[1,1,1]`）
3. %：取余（ $7\%3=1$ ）/ 字符串格式化（`'%d'%5`）

简答 2-3

写出 Python 中 `and` 和 `or` 运算符的短路求值规则，并举例。

参考答案与解析

答案：

- `and`: 从左到右计算，遇到第一个假值就返回该值，否则返回最后一个值。
- `or`: 从左到右计算，遇到第一个真值就返回该值，否则返回最后一个值。

示例：`3 and 0 and 5` 返回 `0`（遇到假值）；`0 or 3 or 5` 返回 `3`（遇到真值）。

编程题

编程题 2-1

编写程序，输入一个三位自然数，计算并输出其百位、十位和个位上的数字。

参考答案与解析

代码：

```
num = int(input("请输入一个三位数："))
hundreds = num // 100
tens = num // 10 % 10
ones = num % 10
print(f"百位：{hundreds}, 十位：{tens}, 个位：{ones}")
```

核心思路：利用整除`//`和取余`%`提取各位数字。

编程题 2-2

编写程序，使用至少两种方法计算 `sum(map(int, str(123456)))` 等价的数字各位之和。

参考答案与解析

方法一（循环）：

```
num = 123456
total = 0
for ch in str(num):
    total += int(ch)
print(total) # 21
```

方法二（map+sum，与题目等价）：

```
print(sum(map(int, str(123456)))) # 21
```

课后作业

作业 2.8

Python 关键字中用来判断两个对象的引用是否相同的是_____。

参考答案与解析

答案：is。is 判断两个变量是否指向同一个内存地址，而 == 仅判断值是否相等。

作业 2.12

Python 关键字_____既可以删除列表中的一个元素，也可以删除整个列表。

参考答案与解析

答案：del。用法：删除指定元素 del list[index]；删除整个列表 del list。

作业 2.13

已知 $x = 3$ ，那么执行语句 $x += 6$ 之后， x 的值为_____。

参考答案与解析

答案：9。 $x += 6$ 为增量赋值，等价于 $x = x + 6$ ，计算得 $3 + 6 = 9$ 。

作业 2.21

表达式 `eval('3*2'+ '22')` 的值为_____。

参考答案与解析

答案：666。第一步字符串拼接：`'3*2'+'22' = '3*222'`；第二步 `eval` 执行表达式： $3 \times 222 = 666$ 。

作业 2.22

表达式 `max(['121','34'])` 的值为_____。

参考答案与解析

答案：'34'。字符串按 ASCII 码逐位比较，字符 '3' 的 ASCII 码大于 '1'，因此结果为 '34'。

作业 2.50

判断对错：表达式 `len(zip([1,2,3],'abcdefg'))` 的值为 3。

参考答案与解析

答案：✓ 正确。`zip` 按最短可迭代对象配对，列表长度为 3，因此结果长度为 3。

作业 2.51

判断对错：语句 `x = input(3)` 的作用是自动输入 3 并赋值给变量 *x*。

参考答案与解析

答案：× 错误。`input()` 的参数为提示文字，并非自动输入的值，需手动输入内容赋值。

第3章 第3章 Python 可迭代对象（序列结构）

知识要点

- 有序序列 vs 无序序列：
 - 有序：列表、元组、字符串——支持下标访问和切片
 - 无序：字典、集合——不支持下标和切片（字典可用键作下标）
- 可变序列 vs 不可变序列：
 - 可变：列表、字典、集合——支持增删改
 - 不可变：元组、字符串——创建后不可修改
- 切片操作（重点）：语法 `seq[start:stop:step]`，左闭右开，支持正负索引
- 列表与元组的区别（重点辨析）：
 - 列表用 `[]` 创建，元组用 `()` 创建（单元素元组需加逗号 `(3,)`）
 - 列表可变（可增删改），元组不可变（核心区别）
 - 元组可作为字典键（因为不可变/可哈希），列表不可
 - 元组比列表更节省内存
 - 列表有更多内置方法（`append`、`insert`、`remove`、`pop` 等）
- 字典（重点）：
 - 键值对无序可变序列，键必须不可变（可哈希）
 - 创建：`{key:value}` 或 `dict()`
 - 访问：`d[key]` 或 `d.get(key, default)`
 - 方法：`keys()`、`values()`、`items()`
 - 使用字典计数（重点应用）、字典推导式
- 集合：无序可变、元素不重复、自动去重，了解集合推导式
- 生成器推导式（了解）：`(x**2 for x in range(10))`——返回生成器对象

- 序列解包：a, b = [1, 2]，左右数量须一致

删除内容：浅复制不考；书上 p80 的 collections 模块内容不考。

经典选择题

选择题 3-1

假设列表 aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]，那么切片 aList[3:7] 的值是（ ）。

- 5, 6, 7, 9
- 6, 7, 9, 11
- 7, 9, 11, 13
- 6, 7, 9, 11, 13

参考答案与解析

答案： B

解析：索引从 0 开始，aList[3] 是 6，aList[7] 是 13 但不包含。切片左闭右开：[3:7] 取索引 3 6，即 [6, 7, 9, 11]。

选择题 3-2

表达式 [3] in [1, 2, 3, 4] 的值为（ ）。

- A. True
- B. False
- 3
- C. 3

参考答案与解析

答案： B

解析：in 判断左侧对象是否为右侧列表的**直接元素**。右侧列表包含元素 1、2、3、4，没有列表 [3]（它不在其中），所以返回 False。

选择题 3-3

列表对象的（ ）方法删除首次出现的指定元素，如果不存在则抛出异常。

- A. pop()
- B. remove()
- C. delete()
- D. clear()

参考答案与解析

答案： B

解析： `remove(x)` 删除首次出现的值 `x`，不存在时抛出 `ValueError`。`pop()` 按索引删除，`del` 按索引或变量删除，`clear()` 清空所有元素。

选择题 3-4

已知 `vec = [[1, 2], [3, 4]]`，则表达式 `[col for row in vec for col in row]` 的值为（ ）。

- [1, 2 , [3, 4]]
- 1, 2, 3, 4
- [1, 3 , [2, 4]]
- (1, 2), (3, 4)

参考答案与解析

答案： B

解析： 嵌套列表推导式，等价于将二维列表展平为一维。先遍历外层 `row`，再遍历内层 `col`，结果 `[1, 2, 3, 4]`。

选择题 3-5

字典和集合的界定符都是（ ）。

- A. []
- B. ()
- C. {}
- D. <>

参考答案与解析

答案： C

解析：字典和集合都用大括号 {}。空字典是 {}，空集合需要 `set()` 创建（因为 {} 被解析为空字典）。

选择题 3-6

使用字典对象的（ ）方法可以返回字典的键值对。

- A. `keys()`
- B. `values()`
- C. `items()`
- D. `get()`

参考答案与解析

答案： C

解析：`items()` 返回键值对视图，`keys()` 返回键视图，`values()` 返回值视图，`get()` 获取指定键的值。

选择题 3-7

假设有 `a = ['name', 'age']` 和 `b = ['Dong', 38]`，一条语句转为字典且以 `a` 为键、`b` 为值，应写作（ ）。

- A. `dict(a, b)`
- B. `dict(zip(a, b))`
- C. `zip(a, b)`
- D. `dict(a=b)`

参考答案与解析

答案：B

解析：`zip(a, b)` 将两个列表按位置配对为元组序列，`dict()` 将元组序列转换为字典，得 `{'name': 'Dong', 'age': 38}`。

选择题 3-8

（ ）可以使用 `del` 命令来删除元组中的部分元素。

- A. 可以
- B. 不可以

参考答案与解析

答案：B

解析：元组是不可变序列，无法删除或修改部分元素，只能删除整个元组对象 `del t`。

选择题 3-9

应从列表尾部进行元素增删操作，因为（ ）。

- A. 代码更简洁
- B. 性能更好（ $O(1)$ ）
- C. 语法要求
- D. 两种情况一样

参考答案与解析

答案： B

解析：列表底层是动态数组（连续内存）。尾部增删 $O(1)$ ；头部/中间增删需要移动后面所有元素， $O(n)$ 效率低。

选择题 3-10

已知 $x = 1:'a', 2:'b', 3:'c'$ ，表达式 $x.keys()$ 的类型是（ ）。

- A. list
- B. tuple
- C. set
- D. dict_keys（视图对象）

参考答案与解析

答案： D

解析： $x.keys()$ 返回的是 `dict_keys` 视图对象，不是列表。可以转换为列表 `list(x.keys())`。该视图会随字典变化而动态反映。

易错概念辨析题

辨析题 3-1

判断对错：列表和元组的主要区别是列表用方括号，元组用圆括号。

参考答案与解析

答案： × 错误（不完全）。

解析：虽然形式上确有方括号/圆括号的区分，但**本质区别**是列表可变、元组不可变。此外，创建单元素元组必须加逗号如 `(3,)`，且元组可作为字典键而列表不行。仅记符号区别会导致理解偏差。

辨析题 3-2

判断对错：`('abc')` 是一个包含字符串 'abc' 的元组。

参考答案与解析

答案：× 错误。

解析：('abc') 等价于 'abc'，括号仅表示运算优先级，不是元组。单元素元组必须加逗号：('abc',) 才是元组。空元组为 ()。

辨析题 3-3

判断对错：列表推导式得到的是生成器对象，生成器表达式得到的是列表。

参考答案与解析

答案：× 错误。

解析：正好相反。列表推导式使用 [] 返回列表；生成器表达式使用 () 返回生成器对象（惰性求值）。例：[x**2 for x in range(3)] 是列表，(x**2 for x in range(3)) 是生成器。

辨析题 3-4

判断对错：字典的键必须是不可变类型，值可以是任意类型。

参考答案与解析

答案：✓ 正确。

解析：字典的键必须可哈希（不可变），如整数、字符串、元组等。列表、字典、集合不可作为键。值可以是任意 Python 对象。

辨析题 3-5

判断对错：集合支持使用索引访问其中的元素。

参考答案与解析

答案：× 错误。

解析：集合是无序序列，不支持下标索引和切片操作。只能通过 in 判断元素是否存在，或遍历访问。

辨析题 3-6

判断对错：切片 aList[0::-1] 和 aList[::-1] 效果相同，都输出整个列表的逆序。

参考答案与解析

答案：× 错误。

解析：`aList[::-1]` 确实输出整个列表的逆序。但 `aList[0::-1]` 是从索引 0 开始逆序取到开头，结果只有一个元素（索引 0 的元素）。两者完全不同。

辨析题 3-7

判断对错：`sorted(list)` 和 `list.sort()` 功能完全相同。

参考答案与解析

答案：× 错误。

解析：`sorted()` 返回新列表，不改变原列表；`sort()` 是原地排序，修改原列表，返回 `None`。`reversed()` 和 `reverse()` 同理。

辨析题 3-8

判断对错：已知 `x = [1, 2, 3]`，执行 `y = x` 后，修改 `y[0] = 9`，则 `x[0]` 也变为 9。

参考答案与解析

答案：✓ 正确。

解析：Python 中变量存储的是引用。`y = x` 使 `y` 指向与 `x` 相同的内存地址（同一个列表对象），所以修改 `y` 会影响 `x`。这与值类型（整数等）的行为不同。

辨析题 3-9

判断对错：空集合可以用 `{}` 创建。

参考答案与解析

答案：× 错误。

解析：`{}` 创建的是空字典。空集合必须用 `set()` 创建。非空集合可以用花括号创建，如 `{1, 2, 3}`。

辨析题 3-10

判断对错：使用 `for` 循环遍历列表并删除元素时，从前向后删和从后向前删效果一样。

参考答案与解析

答案：× 错误。

解析：从前向后删会导致下标错位，可能漏删元素。应该从后向前遍历删除，或者使用列表推导式创建新列表。

简答题

简答 3-1

详细辨析列表和元组的区别（至少四点）。

参考答案与解析

答案：

1. 可变性（最本质）：列表可变，支持增删改；元组不可变，创建后内容不可修改。
2. 创建语法：列表用 `[]`，元组用 `()`。单元素元组须加逗号如 `(3,)`。
3. 内存占用：元组比列表更节省内存，创建速度更快。
4. 可哈希性：元组可作为字典的键和集合的元素（因不可变）；列表不可。
5. 内置方法：列表方法更多（`append`、`insert`、`remove`、`pop`、`sort` 等）；元组方法少（主要用于 `count`、`index`）。

简答 3-2

什么是序列解包？有什么要求？举例说明。

参考答案与解析

答案：序列解包是对多个变量同时赋值。要求等号右侧值的数量与左侧变量数量一致（Python 3 支持用 `*` 收集多余值）。

示例：

```
a, b, c = [1, 2, 3]          # a=1, b=2, c=3
x, *y = [1, 2, 3, 4]        # x=1, y=[2,3,4]
for index, value in enumerate(lst): # 循环中解包
    print(index, value)
```


简答 3-3

为什么应尽量从列表尾部进行元素的增加与删除操作？

参考答案与解析

答案：Python 列表底层是动态数组（连续内存空间）。尾部增删（append/pop）时间复杂度 $O(1)$ ，效率极高。头部或中间插入/删除需要移动后续所有元素，时间复杂度 $O(n)$ ，数据量大时效率很低。从后向前遍历删除元素可以避免下标错位问题。

编程题**编程题 3-1**

编写程序，生成包含 20 个随机整数的列表，然后对其中偶数下标的元素进行降序排列，奇数下标的元素不变（使用切片）。

参考答案与解析

代码：

```
import random
nums = [random.randint(1, 100) for _ in range(20)]
print("Original:", nums)
nums[::2] = sorted(nums[::2], reverse=True)
print("Processed:", nums)
```

解析：切片 `nums[::2]` 取出所有偶数下标元素，排序后赋回原位置。

编程题 3-2

编写程序，生成包含 1000 个 0 100 的随机整数，使用字典统计每个元素的出现次数。

参考答案与解析

代码:

```
import random
nums = [random.randint(0, 100) for _ in range(1000)]
count = {}
for num in nums:
    count[num] = count.get(num, 0) + 1
# 按出现次数降序输出前5个
sorted_items = sorted(count.items(), key=lambda x: x[1], reverse=True)
for num, freq in sorted_items[:5]:
    print(f"数字{num}出现了{freq}次")
```

解析: 使用 dict.get(key, 0) 实现字典计数, sorted() 按值排序。

编程题 3-3

编写程序, 用户输入一个列表和两个整数作为下标, 使用切片获取两个下标之间 (都包含) 的子列表。

参考答案与解析

代码:

```
lst = eval(input("请输入一个列表: "))
i1 = int(input("请输入第一个下标: "))
i2 = int(input("请输入第二个下标: "))
result = lst[i1:i2+1] # +1 使右边界包含
print("子列表为: ", result)
```

解析: 切片左闭右开, 需要 +1 使第二个下标也包含在内。

课后作业

作业 3.1

表达式 `[3] in [1,2,3,4]` 的值为_____。

参考答案与解析

答案: False。in 判断左侧对象是否为右侧列表的直接元素, 右侧无列表元素 [3], 故返回假。

作业 3.3

列表对象的____方法删除首次出现的指定元素，如果列表中不存在要删除的元素，则抛出异常。

参考答案与解析

答案：remove()。用法：list.remove(element)，元素不存在时抛出 ValueError 异常。

作业 3.4

假设列表对象 aList 的值为 [3,4,5,6,7,9,11,13,15,17]，那么切片 aList[3 : 7] 得到的值是____。

参考答案与解析

答案：[6,7,9,11]。Python 列表切片遵循“左闭右开”规则，aList[3:7] 取索引 3 到 6 的元素。

作业 3.5

在 Python 中，字典和集合都是用一对____作为界定符，字典的每个元素由两部分组成，即____和____，其中____不允许重复。

参考答案与解析

答案：{}、键、值、键。字典与集合的界定符均为大括号 {}（空集合需用 set()，空字典为 {}）；字典元素是键值对，键具有唯一性，值可重复。

作业 3.6

使用字典对象的____方法可以返回字典的“键: 值”对，使用字典对象的____方法可以返回字典的所有“键”，使用字典对象的____方法可以返回字典的所有“值”。

参考答案与解析

答案：items()、keys()、values()。示例：d = {1:'a',2:'b'}，d.items() 返回键值对视图，d.keys() 返回键视图，d.values() 返回值视图。

作业 3.7

假设有列表 `a = ['name', 'age', 'sex']` 和 `b = ['Dong', 38, 'Male']`，请使用一条语句将这两个列表的内容转换为字典，并且以列表 `a` 中的元素为”键”，以列表 `b` 中的元素为”值”，这个语句可以写为_____。

参考答案与解析

答案：`dict(zip(a, b))`。`zip(a,b)` 将两个列表按位置配对为元组迭代器，`dict()` 将元组迭代器转换为字典。

作业 3.8

假设有一个列表 `a`，现要求从列表 `a` 中每 3 个元素取 1 个，并且将取到的元素组成新的列表 `b`，可以使用语句_____。

参考答案与解析

答案：`b = a[::3]`。列表切片语法 `[start:end:step]`，步长为 3 表示每隔 2 个元素取 1 个。

作业 3.9

使用列表推导式生成包含 10 个数字 5 的列表，语句可以写为_____。

参考答案与解析

答案：`[5 for _ in range(10)]`。列表推导式循环生成 10 个数字 5，`_` 为占位符变量。

作业 3.10

_____（可以、不可以）使用 `del` 命令来删除元组中的部分元素。

参考答案与解析

答案：不可以。元组是不可变序列，无法修改/删除部分元素，仅能删除整个元组对象。

作业 3.11

已知 `vec = [[1,2],[3,4]]`，则表达式 `[col for row in vec for col in row]` 的值为_____。

参考答案与解析

答案：`[1,2,3,4]`。嵌套列表推导式，遍历二维列表所有元素并展平为一维列表。

作业 3.32

为什么应尽量从列表的尾部进行元素的增加与删除操作？

参考答案与解析

答案：Python 列表为动态数组，底层是连续内存空间：

1. 尾部增删：`append()`、`pop()`，时间复杂度 $O(1)$ ，效率极高；
2. 头部/中间增删：需要移动所有后续元素，时间复杂度 $O(n)$ ，效率极低。

作业 3.28

编写程序，生成包含 1000 个 0 100（包含 0 和 100）的随机整数，并统计每个元素的出现次数。

参考答案与解析

代码：

```
import random
from collections import Counter
nums = [random.randint(0, 100) for _ in range(1000)]
count = Counter(nums)
for num, freq in count.items():
    print(f"数字{num}出现了{freq}次")
```

作业 3.29

编写程序，用户输入一个列表和两个整数作为下标，然后使用切片获取并输出列表中介于两个下标（都包含）之间的元素组成的子列表。

参考答案与解析

代码：

```
lst = eval(input("请输入一个列表："))
index1 = int(input("请输入第一个下标："))
index2 = int(input("请输入第二个下标："))
sub_lst = lst[index1:index2+1]
print("子列表为：", sub_lst)
```

作业 3.30

设计一个字典，并编写程序，用户输入内容作为”键”，然后输出字典中对应的”值”，如果用户输入的”键”不存在，则输出”您输入的键不存在！”。

参考答案与解析

代码：

```
info = {"姓名": "Dong", "年龄": 38, "性别": "Male"}
key = input("请输入要查询的键: ")
if key in info:
    print(f"对应的值为: {info[key]}")
else:
    print("您输入的键不存在！")
```

作业 3.31

编写程序，生成包含 20 个随机数的列表，然后将前 10 个元素升序排列，后 10 个元素降序排列，并输出结果。

参考答案与解析

代码：

```
import random
nums = [random.randint(1, 100) for _ in range(20)]
nums[:10] = sorted(nums[:10])
nums[10:] = sorted(nums[10:], reverse=True)
print("处理后的列表: ", nums)
```

第 4 章 第 4 章程序控制结构

知识要点

- 条件表达式：只要不是 False、0、None、空序列等，均视为 True
- 关系运算符连用： $1 < x < 10$ （等价于 $1 < x \text{ and } x < 10$ ）
- 逻辑运算符短路求值（同第 2 章）
- 选择结构：
 - 单分支：if 条件：代码块
 - 双分支：if-else
 - 多分支：if-elif-else
 - 嵌套选择结构
- 循环结构：
 - for 变量 in 可迭代对象：循环
 - while 条件：循环
 - range() 在循环中的使用
 - break：提前结束当前循环；continue：提前结束本次循环
- 带 else 子句的循环结构：
 - for-else 和 while-else
 - 循环正常结束（没有被 break 打断）时执行 else 块
 - 如果循环被 break 打断，else 块不执行
- 代码规范：条件表达式后必须有冒号，代码块必须缩进

经典选择题

选择题 4-1

下面代码的运行结果为（ ）。

```
for i in range(3):  
    print(i, end=',')
```

- A. 0,1,2
- B. 0,1,2,
- C. 1,2,3
- D. 0,1,2,3,

参考答案与解析

答案： B

解析： range(3) 生成 0,1,2，三次 print 都带 end=',', 所以输出 0,1,2,。

选择题 4-2

下面代码的运行结果为（ ）。

```
for n in range(100, 1, -1):  
    for i in range(2, n):  
        if n % i == 0:  
            break  
    else:  
        print(n)  
        break
```

- A. 2
- B. 97
- C. 99
- D. 100

参考答案与解析

答案： B

解析：外层从 100 向下递减。内层 for-else 结构：如果内层 for 被 break（说明 n 有因数，非素数），else 不执行；如果内层 for 正常结束（n 是素数），else 执行，输出 n 并 break 外层。从 100 向下第一个素数是 97。

选择题 4-3

下面选项中作为条件表达式时与 True 等价的是（ ）。(多选)

A. $3 + 5$

3

B. 0

参考答案与解析

答案： AC

解析：非零数值 (A) 和非空列表 (C) 在条件判断中等价于 True。空列表 [] 和 0 等价于 False。

选择题 4-4

break 语句的作用是（ ）。

A. 结束本次循环，进入下一次循环

B. 提前结束所在循环结构

C. 结束整个程序

D. 跳过当前语句块

参考答案与解析

答案： B

解析：break 提前结束所在循环；continue 仅结束本次循环进入下一次迭代。

易错概念辨析题

辨析题 4-1

判断对错：在 for-else 结构中，不管循环是否被 break 打断，else 块都会执行。

参考答案与解析

答案：× 错误。

解析：else 块仅在循环正常结束（未被 break 打断）时才执行。如果循环被 break 打断，else 块不执行。

辨析题 4-2

判断对错：条件表达式 `3 > 5 and a > b`（a 和 b 未定义）会因为 NameError 而出错。

参考答案与解析

答案：× 错误。

解析：由于短路求值，`3 > 5` 为 False，and 不再计算右侧表达式，直接返回 False。右侧的 `a > b` 不会被执行，因此不会抛出异常。

辨析题 4-3

判断对错：`if x = 5:` 是正确的 Python 条件语句。

参考答案与解析

答案：× 错误。

解析：条件表达式中不允许使用赋值运算符 `=`，必须使用比较运算符 `==`。正确写法：`if x == 5:`。

简答题

简答 4-1

简述 for-else 和 while-else 结构中 else 的执行条件。

参考答案与解析

答案：else 块在循环正常结束时执行（即循环条件变为 False，或可迭代对象被完全遍历）。如果循环被 break 语句提前结束，则 else 不执行。经典应用场景：检查素数——若内层循环遍历完所有除数都没有找到因数（未被 break），则说明是素数。

简答 4-2

简述 break 和 continue 的区别。

参考答案与解析

答案：break 彻底退出当前循环（不执行 else 块）。continue 仅跳过本次循环的剩余代码，进入下一次迭代。两者都只影响最内层循环。

编程题

编程题 4-1

编写程序，判断用户输入的年份是否为闰年。闰年条件：能被 400 整除，或能被 4 整除但不能被 100 整除。

参考答案与解析

代码：

```
year = int(input("请输入一个4位年份："))
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
    print(f"{year}是闰年")
else:
    print(f"{year}不是闰年")
```

编程题 4-2

编写程序，生成包含 50 个随机整数的列表，删除其中所有奇数（从后向前删）。

参考答案与解析

代码：

```
import random
nums = [random.randint(1, 100) for _ in range(50)]
print("Original:", nums)
for i in range(len(nums)-1, -1, -1):
    if nums[i] % 2 == 1:
        del nums[i]
print("After removal:", nums)
```

解析：从后向前遍历避免因删除元素导致下标错位遗漏元素。

编程题 4-3

至少使用三种方法计算 100 以内所有奇数的和。

参考答案与解析

代码：

```
# 方法一：range步长
s1 = sum(range(1, 100, 2))

# 方法二：for循环
s2 = 0
for i in range(1, 100, 2):
    s2 += i

# 方法三：列表推导式
s3 = sum([i for i in range(100) if i % 2 == 1])

# 方法四：while循环
s4, n = 0, 1
while n < 100:
    s4 += n
    n += 2

print(s1, s2, s3, s4) # 均为2500
```

课后作业

作业 4.1

已知 $x = [1, 1]$ ，那么执行完以下语句后，表达式 $x[-1]$ 的值为_____。

```
for i in range(5):
    x.append(x[-1]+x[-2])
```

参考答案与解析

答案：13。该循环生成斐波那契数列：初始 $x = [1, 1]$ ，经过 5 次迭代依次追加 2, 3, 5, 8, 13，最终 $x = [1, 1, 2, 3, 5, 8, 13]$ ，因此 $x[-1] = 13$ 。

作业 4.2

下面代码的运行结果为_____。

```
for index, (f, s) in enumerate(zip((1, 2, 3), [4, 5])):
    print(f, end=',')
```

参考答案与解析

答案：1,2,。zip((1,2,3), [4,5]) 按最短长度配对，得到 (1,4) 和 (2,5)。enumerate 遍历后每次输出第一个元素 f，以逗号结尾，故输出 1,2,。

作业 4.3

下面代码的运行结果为_____。

```
for i in range(3):
    print(i, end=',')
```

参考答案与解析

答案：0,1,2,。range(3) 生成 0,1,2，循环依次输出并以逗号结尾。

作业 4.4

下面代码的运行结果为_____。

```
for n in range(100, 1, -1):
    for i in range(2, n):
        if n % i == 0:
            break
    else:
        print(n)
        break
```

参考答案与解析

答案：97。外层循环从 100 向下递减，内层循环检查 n 是否为素数。从 100 开始向下找到的第一个素数是 97。

作业 4.11

编写程序，运行后用户输入 4 位整数作为年份，判断其是否为闰年。

参考答案与解析

代码:

```
year = int(input("Please enter a 4-digit year: "))
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
    print(f"{year} is a leap year")
else:
    print(f"{year} is not a leap year")
```

作业 4.12

编写程序，生成一个包含 50 个随机整数的列表，然后删除其中所有奇数（提示：从后向前删）。

参考答案与解析

代码:

```
import random
nums = [random.randint(1, 100) for _ in range(50)]
print("Original list:", nums)
for i in range(len(nums) - 1, -1, -1):
    if nums[i] % 2 == 1:
        del nums[i]
print("After removing odd numbers:", nums)
```

作业 4.13

编写程序，生成一个包含 20 个随机整数的列表，然后对其中偶数下标的元素进行降序排列，奇数下标的元素不变（提示：使用切片）。

参考答案与解析

代码:

```
import random
nums = [random.randint(1, 100) for _ in range(20)]
print("Original list:", nums)
nums[::2] = sorted(nums[::2], reverse=True)
print("After processing:", nums)
```

作业 4.15

编写程序，至少使用三种不同的方法计算 100 以内所有奇数的和。

参考答案与解析

代码：

```
# Method 1: use range step
sum1 = sum(range(1, 100, 2))
print("Method 1 (range step):", sum1)

# Method 2: use loop with condition
sum2 = 0
for i in range(1, 100):
    if i % 2 == 1:
        sum2 += i
print("Method 2 (loop):", sum2)

# Method 3: use list comprehension
sum3 = sum([i for i in range(100) if i % 2 == 1])
print("Method 3 (list comprehension):", sum3)
```

三种方法结果均为 2500。

作业 4.16

编写程序，输出所有由 1、2、3、4 这 4 个数字组成的素数，并且在每个素数中每个数字只使用一次。

参考答案与解析

代码:

```
import itertools
import math

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

result = []
for perm in itertools.permutations([1, 2, 3, 4]):
    num = 1000 * perm[0] + 100 * perm[1] + 10 * perm[2] + perm[3]
    if is_prime(num):
        result.append(num)

print("Primes composed of 1,2,3,4:", sorted(result))
```

运行结果为 [1423, 2143, 2341, 4231]。

第 5 章 第 5 章函数

知识要点

- 函数定义: `def 函数名 (参数列表):`, 必须有冒号和缩进
- 函数参数 (重点):
 - 位置参数、默认值参数、关键参数
 - `*args`: 收集多余位置参数为元组
 - `**kwargs`: 收集多余关键字参数为字典
 - 传递参数时的序列解包: `func(*list)` 和 `func(**dict)`
- 形参与实参 (重点):
 - 形参是函数定义时的参数名, 实参是调用时传入的值
 - Python 传递参数是传引用 (实参的引用传给形参)
 - 如果实参是可变序列 (如列表), 在函数内直接修改其元素会影响外部实参
 - 如果实参是不可变类型 (如整数), 函数内重新赋值不影响外部实参
- 变量作用域 (重点):
 - 函数内只引用变量不赋值 → 隐式全局变量
 - 函数内对变量赋值 → 隐式局部变量 (除非用 `global` 声明)
 - 使用 `global` 关键字在函数内定义或修改全局变量
- 默认值参数的陷阱: 默认值只在函数定义时计算一次, 如果是可变对象 (如 `[]`), 多次调用会累积
- `lambda` 表达式: 创建匿名函数, 如 `lambda x: x**2`
- `yield` 语句 (了解): 使函数成为生成器函数, 返回生成器对象, 支持惰性求值

删除内容: 递归调用不考。第 5 节 (yield 相关内容) 仅需知道 `yield` 语句的作用, 不过多要求。

经典选择题

选择题 5-1

在函数内部可以通过关键字（ ）来定义全局变量。

- A. self
- B. this
- C. global
- D. static

参考答案与解析

答案： C

解析： `global x` 声明 `x` 为全局变量，函数内对其的修改会影响外部。

选择题 5-2

如果函数中没有 `return` 语句，那么该函数的返回值是（ ）。

- A. 0
- B. False
- C. None
- D. 抛出异常

参考答案与解析

答案： C

解析： Python 函数无 `return` 或执行不带值的 `return` 时，默认返回 `None`。

选择题 5-3

下面代码的运行结果为（ ）。

```
def func(para):  
    para = 3  
n = 5  
func(n)  
print(n)
```

- A. 3
- B. 5
- C. None
- D. 抛出异常

参考答案与解析

答案：B

解析：n 是整数（不可变），函数内 `para = 3` 将局部变量 `para` 指向新对象，不影响外部 `n` 的值。

选择题 5-4

下面代码的运行结果为（ ）。

```
def func(para):  
    para[0] = 3  
para = [5]  
func(para)  
print(para)
```

- 5
- 3
- A. 3
 - B. 5

参考答案与解析

答案： B

解析： para 是列表（可变序列），函数内修改元素 `para[0]=3` 直接影响外部实参。

选择题 5-5

已知函数定义 `def func(a, b, *p): pass`，通过 `func(1, 2, 3, 4, 5)` 调用，形参 p 的值为（ ）。

- A. 4
- B. 5
- C. (3, 4, 5)

3, 4, 5

参考答案与解析

答案： C

解析： *p 收集多余的位置参数为元组。1 和 2 分配给 a 和 b，剩余 3,4,5 收集为元组 (3, 4, 5)。

选择题 5-6

下面代码的运行结果为（ ）。

```
x = 3
def modify():
    x = 5
modify()
print(x)
```

- A. 3
- B. 5
- C. 0
- D. 抛出异常

参考答案与解析

答案： A

解析：函数内 `x = 5` 创建了新的局部变量 `x`，不影响外部全局变量 `x`。若要在函数内修改全局变量，需用 `global x` 声明。

选择题 5-7

包含（ ）语句的函数可以用来创建生成器对象。

- A. `return`
- B. `yield`
- C. `global`
- D. `break`

参考答案与解析

答案： B

解析：含 `yield` 语句的函数是生成器函数，调用返回生成器对象，支持惰性求值和 `next()` 逐步取值。

易错概念辨析题**辨析题 5-1**

判断对错：调用带有默认值参数的函数时，不能为默认值参数传递任何值，必须使用函数定义时设置的默认值。

参考答案与解析

答案： × 错误。

解析：可以为默认值参数传递新值，传入的值会覆盖默认值。

辨析题 5-2

判断对错：在函数内部没有任何声明的情况下直接为某个变量赋值，这个变量一定是函数内部的局部变量。

参考答案与解析

答案：✓ 正确。

解析：函数内部赋值操作默认创建局部变量。只有使用 `global` 声明后，赋值才影响全局变量。

辨析题 5-3

判断对错：以下代码运行正常，输出 3。

```
def func():  
    print(x)  
x = 3  
func()
```

参考答案与解析

答案：✓ 正确。

解析：函数内只引用 `x` 的值而未对其赋值，`x` 是隐式全局变量，可以正常访问。如果函数内先赋值再引用则会出错。

辨析题 5-4

判断对错：以下代码输出是 `[3, 3, 3]`。

```
def func(para=[]):  
    para.append(3)  
    return para  
func()  
func()  
print(func())
```

参考答案与解析

答案：✓ 正确。

解析：默认值参数只在函数定义时计算一次。三次调用共享同一个列表对象，第一次追加 3 得 `[3]`，第二次追加得 `[3,3]`，第三次追加后返回 `[3,3,3]`。

辨析题 5-5

判断对错：`lambda` 表达式只能用来创建匿名函数，不能起名字。

参考答案与解析

答案： × 错误。

解析： lambda 表达式虽然常用于匿名函数，但可以赋值给变量为其起名，如 `f = lambda x: x**2`。不过更推荐用 `def` 定义具名函数。

辨析题 5-6

判断对错：已知不同的三个函数 A、B、C，在函数 A 中调用 B，函数 B 中又调用了 C，这种调用方式称作递归调用。

参考答案与解析

答案： × 错误。

解析： 这是函数之间的普通嵌套调用。递归调用是指一个函数直接或间接地调用自身。

辨析题 5-7

判断对错：生成器函数的调用结果是一个确定的值。

参考答案与解析

答案： × 错误。

解析： 生成器函数调用返回一个生成器对象（迭代器），不是确定值。需要通过 `next()` 或 `for` 循环逐步获取值，每次调用产生一个值并暂停。

简答题

简答 5-1

分别举例说明，什么情况下在函数内部修改形参会影响到外部实参，什么情况下不会。

参考答案与解析

答案：

- **会影响：**实参是可变序列（如列表、字典），在函数内直接修改其元素。如 `para[0]=3` 会改变外部列表。
- **不会影响：**实参是不可变类型（整数、字符串、元组），在函数内重新赋值（如 `para = 3`）只是将形参指向新对象，不影响外部变量。

本质原因：Python 传引用。可变对象可通过引用修改内容；不可变对象只能重新创建，不影响原引用指向的对象。

简答 5-2

简述局部变量和全局变量的判定规则。

参考答案与解析

答案：

1. 函数内只引用某个变量而不对其赋值 → 该变量是隐式全局变量（自动向上查找）。
2. 函数内任意位置有为变量赋新值的操作 → 该变量被认为是隐式局部变量，即使赋值在引用之后也会导致前面引用报错。
3. 若要在函数内修改全局变量 → 必须使用 `global` 关键字显式声明。

简答 5-3

解释 `*args` 和 `**kwargs` 的含义和用法。

参考答案与解析

答案：

- ***args：**收集多余的位置参数为元组。形参名前加一个星号。如 `def f(a, *args):` 调用 `f(1,2,3)` 则 `a=1, args=(2,3)`。
- ****kwargs：**收集多余的关键字参数为字典。形参名前加两个星号。如 `def f(**kwargs):` 调用 `f(x=1,y=2)` 则 `kwargs={'x':1, 'y':2}`。

编程题

编程题 5-1

编写函数，判断一个整数是否为素数，并编写主程序调用。

参考答案与解析

代码：

```
import math

def is_prime(n):
    """判断n是否为素数"""
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

if __name__ == '__main__':
    num = int(input("请输入一个整数: "))
    if is_prime(num):
        print(f"{num}是素数")
    else:
        print(f"{num}不是素数")
```

解析：只需检查到 $\sqrt{n}$ ，大幅减少循环次数。

编程题 5-2

编写函数，接收一个字符串，分别统计大写字母、小写字母、数字、其他字符的个数，以元组形式返回。

参考答案与解析

代码:

```
def count_chars(s):
    upper = lower = digit = other = 0
    for ch in s:
        if ch.isupper():
            upper += 1
        elif ch.islower():
            lower += 1
        elif ch.isdigit():
            digit += 1
        else:
            other += 1
    return (upper, lower, digit, other)

print(count_chars("Hello, Python 123!"))
```

编程题 5-3

编写函数，接收任意多个整数并返回其中的最大值和所有整数之和。

参考答案与解析

代码:

```
def max_and_sum(*args):
    if not args:
        return (None, 0)
    return (max(args), sum(args))

print(max_and_sum(1, 5, 3, 9, 2)) # (9, 20)
```

解析：使用 *args 接收任意多个参数。

课后作业

作业 5.1

在函数内部可以通过关键字____ 来定义全局变量。

参考答案与解析

答案：global。在函数中使用 `global x` 声明，即可将 `x` 定义为全局变量。

作业 5.2

如果函数中没有 `return` 语句或者 `return` 语句但不带任何返回值，那么该函数的返回值为_____。

参考答案与解析

答案：None。Python 中函数如果没有执行 `return` 语句或执行了不带值的 `return`，默认返回 `None`。

作业 5.4

下面代码的运行结果为_____。

```
def func(para):  
    para = 3  
n = 5  
func(n)  
print(n)
```

参考答案与解析

答案：5。Python 中整数是不可变类型，函数内 `para = 3` 将局部变量 `para` 指向新对象 3，外部变量 `n` 仍指向 5，互不影响。

作业 5.31

判断对错：定义 Python 函数时，如果函数中没有 `return` 语句，则默认返回空值 `None`。

参考答案与解析

答案：✓ 正确。如 5.2 所述，函数无 `return` 语句时默认返回 `None`。

作业 5.34

判断题：调用带有默认值参数的函数时，不能为默认值参数传递任何值，必须使用函数定义时设置的默认值。

参考答案与解析

答案：× 错误。调用函数时可以为默认值参数传递新值，传入的值会覆盖默认值。

作业 5.37

编写函数，判断一个整数是否为素数，并编写主程序调用该函数。

参考答案与解析

代码：

```
import math

def is_prime(n):
    """Check if positive integer n is prime, return True if prime else False"""
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

# Main program
num = int(input("Please enter an integer: "))
if is_prime(num):
    print(f"{num} is prime")
else:
    print(f"{num} is not prime")
```

判断素数只需检查 2 到 $\sqrt{n}$ 之间是否有因数，可大幅减少循环次数。

作业 5.38

编写函数，接收一个字符串，分别统计大写字母、小写字母、数字、其他字符的个数，并以元组的形式返回结果。

参考答案与解析

代码:

```
def count_chars(s):
    upper = lower = digit = other = 0
    for ch in s:
        if ch.isupper():
            upper += 1
        elif ch.islower():
            lower += 1
        elif ch.isdigit():
            digit += 1
        else:
            other += 1
    return (upper, lower, digit, other)

# Test
result = count_chars("Hello, Python 123!")
print(result) # Output like (2, 11, 3, 4)
```

第 6 章 第 6 章面向对象程序设计

知识要点

- 面向对象三要素：封装、继承、多态（必须掌握概念）
 - 封装：将数据和对数据的操作封装在类内部，对外隐藏实现细节
 - 继承：子类（派生类）继承父类（基类）的属性和方法，支持代码复用
 - 多态：不同对象对同一方法做出不同响应
- 类的定义：`class 类名 (基类):`，使用 `self` 指代实例本身
- 构造方法：`__init__(self)` 在创建对象时自动调用
- 析构方法：`__del__(self)` 在对象销毁时调用
- 下划线命名规则（第 9 章也用到）：
 - `__xxx__`：系统预定义的特殊方法（如 `__init__`）
 - `_xxx`：保护成员（protected），约定内部使用，但外部可访问
 - `__xxx`：私有成员（private），触发名称修饰为 `_ 类名 __xxx`
- 私有成员：双下划线开头不以双下划线结尾，外部不能直接访问，可通过对象名 `_ 类名 __` 成员名间接访问（不推荐）
- 类数据成员共享性：类属性被所有实例共享
- 方法与函数的区别：方法通过对象/类调用，有 `self` 参数；函数独立调用
- `super()`：调用父类方法
- 重点复习作业题：作业中的面向对象编程题

经典选择题

选择题 6-1

面向对象程序设计的三要素是（ ）。

- A. 类、对象、方法
- B. 封装、继承、多态
- C. 数据、方法、接口
- D. 公有、私有、保护

参考答案与解析

答案：B

解析：封装、继承、多态是面向对象的三大核心特征。

选择题 6-2

在 Python 中，不论类的名字是什么，构造方法的名称固定为（ ）。

- A. init
- B. `__init__`
- C. constructor
- D. `__new__`

参考答案与解析

答案：B

解析：`__init__()` 是 Python 类的构造方法，创建对象时自动调用。`__new__()` 在 `__init__()` 之前被调用，用于创建实例。

选择题 6-3

假设 a 为类 A 的对象且包含私有数据成员 `__value`，在类外部间接访问的正确写法是（ ）。

- A. `a.__value`
- B. `a._value`
- C. `a._A__value`
- D. `A.__value`

参考答案与解析

答案： C

解析： Python 对私有成员进行名称修饰（name mangling），`__value` 在类 A 中实际被重命名为 `_A__value`。外部可通过此名称访问，但不推荐。

选择题 6-4

在 Python 中，私有成员以（ ）开头。

- A. 单下划线
- B. 双下划线且不以双下划线结尾
- C. 双下划线开头和结尾
- D. 任意下划线

参考答案与解析

答案： B

解析： 以两个或更多下划线开头且不以两个或更多下划线结尾表示私有成员。双下划线开头和结尾（如 `__init__`）是特殊方法。单下划线开头是保护成员。

易错概念辨析题**辨析题 6-1**

判断对错：在面向对象程序设计中，函数和方法是完全一样的，都必须为所有参数进行传值。

参考答案与解析

答案：× 错误。

解析：函数和方法不同。方法是定义在类中的函数，实例方法的第一个参数为 `self`，通过对象调用时自动传递，不需手动传值。方法是依赖于类或对象的，函数是独立的。

辨析题 6-2

判断对错：在派生类中不能直接访问基类的私有成员。

参考答案与解析

答案：✓ 正确。

解析：私有成员（双下划线开头）经过名称修饰，在派生类中不能直接访问。派生类可以访问基类的公有成员和保护成员（单下划线开头）。

辨析题 6-3

判断对错：如果在派生类中没有定义构造方法，会自动继承基类的构造方法。

参考答案与解析

答案：✓ 正确。

解析：派生类未定义 `__init__` 时，会继承基类的构造方法。如果派生类定义了构造方法，则不会自动调用基类的构造方法，需用 `super().__init__()` 手动调用。

辨析题 6-4

判断对错：所有实例方法都必须至少有一个名为 `self` 的参数，且必须是第一个参数。

参考答案与解析

答案：✓ 正确。

解析：实例方法的第一个参数必须表示对象本身，约定用 `self` 命名。类方法用 `cls`，静态方法不需要特殊参数。

辨析题 6-5

判断对错：以下画线开头的变量名 `_xxx` 是私有成员，外部绝对不能访问。

参考答案与解析

答案：× 错误。

解析：单下划线开头 `_xxx` 是保护成员（protected），是约定而非强制约束，外部仍可访问。双下划线开头 `__xxx` 才是私有成员（触发名称修饰）。注意区分单下划线和双下划线的本质不同。

简答题

简答 6-1

简单解释 Python 中以下画线开头和结束的变量名的含义。

参考答案与解析

答案：

1. `_xxx`：单下划线开头——保护成员（protected），建议在类内部和子类中使用，外部应避免直接访问。
2. `__xxx`：双下划线开头——私有成员（private），触发名称修饰（Name Mangling），实际被重命名为 `_ 类名 __xxx`。
3. `__xxx__`：双下划线开头和结尾——系统预定义的特殊方法（魔术方法），如 `__init__`、`__str__` 等，不支持自由定义和添加。

简答 6-2

简述封装、继承、多态的概念，并各举一个生活中的例子。

参考答案与解析

答案：

- **封装**：把数据和方法包装在类中，隐藏内部细节，只暴露接口。如：手机把复杂的电路和芯片封装在机身内部，用户只需通过屏幕按钮操作。
- **继承**：子类继承父类的属性和方法。如：电动汽车继承汽车的基本功能（方向盘、刹车等），并增加电池管理的特性。
- **多态**：同一方法在不同对象中有不同表现。如：同样是“叫”的方法，狗会“汪汪叫”，猫会“喵喵叫”。

编程题

编程题 6-1

编写类 Person，包含 name 和 age 属性及 info() 方法显示信息。继承 Person 创建 Student 类，添加 major 属性和 set_major() 方法。

参考答案与解析

代码：

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"姓名: {self.name}, 年龄: {self.age}")

class Student(Person):
    def __init__(self, name, age, major=""):
        super().__init__(name, age)
        self.major = major

    def set_major(self, major):
        self.major = major

    def info(self):
        super().info()
        print(f"专业: {self.major}")

stu = Student("张三", 20)
stu.set_major("计算机科学")
stu.info()
```

解析：使用 super() 调用父类构造方法，info() 方法覆盖实现多态。

第7章 第7章字符串

知识要点

- 字符串：不可变有序序列，用单引号/双引号/三引号作定界符
- 转义字符（了解）：\n、\t、\\等
- 常用方法：
 - find(sub)/index(sub)：查找子串位置，find 不存在返回-1，index 不存在抛异常
 - count(sub)：统计子串出现次数
 - split(sep)：分割字符串为列表
 - join(iterable)：将可迭代对象的元素用分隔符连接为字符串，如'/''.join(['a','b','c'])
→ 'a/b/c'
 - strip()/lstrip()/rstrip()：去除两侧/左侧/右侧指定字符
 - lower()/upper()：转小写/大写
 - startswith()/endswith()：判断开头/结尾
 - replace(old, new)：替换子串
- 字符串运算符：+ 连接、* 重复、in 成员测试、[] 索引、[:] 切片
- % 运算符格式化（了解）

删除内容：7.4.5 节的三个函数（对标课本的 ljust/rjust/zfill 等排版函数，以课件中标注为准）不考。

经典选择题

选择题 7-1

已知列表 `x = ['11', '2', '3']`，则表达式 `max(x)` 的值为（ ）。

- A. '11'
- B. '2'
- C. '3'
- D. 11

参考答案与解析

答案：C

解析：字符串比较按字典序逐位比较 ASCII 码。`'1'`(ASCII 49) < `'2'`(50) < `'3'`(51)，故最大值为 `'3'`。

选择题 7-2

表达式 `len('::'.join(['a', 'b', 'c']))` 的值为（ ）。

- A. 5
- B. 6
- C. 7
- D. 3

参考答案与解析

答案：C

解析：`'::'.join(['a', 'b', 'c'])` 返回 `'a::b::c'`，该字符串包含 3 个字母和 4 个冒号 (`a + :: + b + :: + c`)，长度 $3 + 4 = 7$ 。

选择题 7-3

表达式 `'a' + 'b'` 的值为 ()。

- A. `'ab'`
- B. `'a+b'`
- C. `ab` (数值)
- D. 抛出异常

参考答案与解析

答案：A

解析：`+` 用于字符串时执行连接操作，`'a'+'b'` 得 `'ab'`。

选择题 7-4

表达式 `'Beautiful is better than ugly.'.index('beautiful')` 的结果是 ()。

- A. 0
- B. -1
- C. 1
- D. 引发 `ValueError` 异常

参考答案与解析

答案：D

解析：`index()` 区分大小写，原字符串中是 `'Beautiful'` (大写 B)，而搜索的是 `'beautiful'` (小写 b)，找不到则抛出 `ValueError` 异常。`find()` 在找不到时返回 -1。

易错概念辨析题**辨析题 7-1**

判断对错：字符串属于可变序列，可以通过下标修改其中的字符。

参考答案与解析

答案：× 错误。

解析：字符串是不可变序列，不能直接修改其中的字符。如 `s[0]='H'` 会报错。所有字符串方法都返回新字符串，不修改原字符串。

辨析题 7-2

判断对错：`split()` 和 `join()` 作用相反——`split` 将字符串拆分为列表，`join` 将列表拼接为字符串。

参考答案与解析

答案：✓ 正确。

解析：`'a,b,c'.split(',')` 得 `['a','b','c']`；`','.join(['a','b','c'])` 得 `'a,b,c'`。两者互为逆操作。

辨析题 7-3

判断对错：`find()` 和 `index()` 功能完全相同。

参考答案与解析

答案：× 错误。

解析：两者都查找子串位置，但找不到时行为不同：`find()` 返回 -1，`index()` 抛出 `ValueError` 异常。

辨析题 7-4

判断对错：字符串的 `strip()` 方法会修改原字符串。

参考答案与解析

答案：× 错误。

解析：字符串是不可变类型，`strip()` 返回新字符串，不修改原字符串。所有字符串方法都如此。

简答题

简答 7-1

简述 `join()` 方法的用法和注意点，举一个例子。

参考答案与解析

答案: `join()` 是字符串方法, 用于连接可迭代对象中的元素。

语法: ' 分隔符'.`join()`(可迭代对象)

示例: `'/'.join(['usr','local','bin'])` → `'usr/local/bin'`

注意: 可迭代对象中的元素必须都是字符串类型, 否则会抛出 `TypeError` 异常。`join()` 的调用者是分隔符字符串, 而非被连接的序列。

编程题

编程题 7-1

编写程序, 将一段英文中单独的字母 `i` 误写纠正为 `I` (单独的字母 `i` 指作为独立单词的 `i`)。

参考答案与解析

代码:

```
import re
text = "i am a student, and i like python."
corrected = re.sub(r'\bi\b', 'I', text)
print("Before:", text)
print("After: ", corrected)
```

解析: `\bi\b` 匹配单词边界内的单独字母 `i`, 将其替换为 `I`。

第 8 章 第 8 章文件操作

知识要点

- 文本文件与二进制文件概念（了解）
 - 文本文件：以字符为单位存储，可用文本编辑器直接阅读（.txt/.py/.csv 等）
 - 二进制文件：以字节为单位存储，需专门软件解析（.jpg/.mp4/.exe 等）
- 文件操作三步骤：打开文件 → 操作文件 → 关闭文件
- 文件打开模式：
 - 'r'：只读（文件不存在则报错）
 - 'w'：只写（文件已存在则清空）
 - 'a'：追加（文件指针在尾部）
 - 'r+'：读写（文件必须存在）
- 常用方法：read()/readline()/readlines()/write()/writelines()/close()
- 上下文管理 with 语句（必须掌握）：
 - 语法：with open(file, mode) as f: 操作
 - 优点：自动管理文件对象，退出 with 块时自动调用 close()，即使发生异常也能保证文件被正确关闭
- os.path.exists()：判断文件或路径是否存在

删除内容：二进制文件不考；t（文本模式）和 b（二进制模式）的具体操作不考；第 8 章正则表达式不考。

经典选择题

选择题 8-1

Python 内置函数（ ）用来打开或创建文件并返回文件对象。

- A. file()
- B. open()
- C. read()
- D. fopen()

参考答案与解析

答案： B

解析： `open()` 是 Python 内置函数，用于打开或创建文件，返回文件对象。

选择题 8-2

使用上下文管理关键字（ ）可以自动管理文件对象。

- A. with
- B. try
- C. for
- D. if

参考答案与解析

答案： A

解析： `with open(...) as f:` 是上下文管理语句，退出时自动关闭文件。

选择题 8-3

使用 `open()` 以 'w' 模式打开文件时，以下描述正确的是（ ）。

- A. 只能读取内容
- B. 只能写入内容，文件已存在则清空
- C. 可以读写
- D. 追加写入

参考答案与解析

答案： B

解析： 'w' 是只写模式，若文件已存在则自动清空原有内容；文件不存在则创建新文件。

易错概念辨析题

辨析题 8-1

判断对错：读写文件时，只要程序中调用了文件对象的 `close()` 方法，就一定可以保证文件被正确关闭。

参考答案与解析

答案： × 错误。如果在调用 `close()` 之前程序因异常而终止，则 `close()` 不会被执行。推荐使用 `with` 语句自动管理文件对象。

课后作业

作业 9.2

Python 内置函数_____用来打开或创建文件并返回文件对象。

参考答案与解析

答案： `open()`。`open(file, mode='r', ...)` 是 Python 内置函数，用于打开或创建文件，返回文件对象。

作业 9.5

使用上下文管理关键字_____可以自动管理文件对象，不论何种原因结束该关键字中的语句块，都能保证文件被正确关闭并且已写入的内容确实保持到硬盘上。

参考答案与解析

答案：with。with open(...) as f: 语句块结束后自动调用 f.close()，确保文件被正确关闭。

作业 9.24

判断对错：内置函数 open() 使用 'r' 模式打开的文件，只能读取其中的内容，不能写入任何新内容。

参考答案与解析

答案：✓ 正确。'r' 为只读模式，只能读取文件内容，写入会抛出 io.UnsupportedOperation 异常。

作业 9.26

判断对错：读写文件时，只要程序中调用了文件对象的 close() 方法，就一定可以保证文件被正确关闭。

参考答案与解析

答案：× 错误。如果在调用 close() 之前程序因异常而终止，则 close() 不会被执行，文件可能未正确关闭。推荐使用 with 语句自动管理文件对象。

作业 9.42

假设当前目录下有一个英文文本文件 example.txt，编写程序逐行读取其内容，并将其中的大写字母变为小写字母，小写字母变为大写字母，输出原始内容和处理后的内容。

参考答案与解析

代码：

```
with open('example.txt', 'r', encoding='utf-8') as f:
    lines = f.readlines()

for i, line in enumerate(lines, 1):
    swapped = line.swapcase()
    print(f"Line {i} original: {line.rstrip()}")
    print(f"Line {i} swapped: {swapped.rstrip()}")
    print()
```

使用字符串的 `swapcase()` 方法可直接实现大小写互换。

作业 9.44

简单解释文本文件与二进制文件的区别。

参考答案与解析

答案：文本文件与二进制文件的主要区别如下：

1. **存储方式**：文本文件以字符为单位存储，每个字符对应一个编码（如 ASCII、UTF-8）；二进制文件以字节为单位直接存储数据。
2. **可读性**：文本文件可以使用记事本等文本编辑器直接打开和阅读；二进制文件通常需要专门的软件才能解析和显示。
3. **操作方式**：Python 中使用 `open()` 时，文本模式（'r'、'w'）会自动处理编码和行尾符；二进制模式（'rb'、'wb'）不对数据进行任何转换。
4. **编码参数**：文本模式可以指定 `encoding` 参数；二进制模式不能指定 `encoding`。
5. **常见类型**：文本文件如 `.txt`、`.py`、`.csv`、`.html` 等；二进制文件如图片（`.jpg`、`.png`）、视频（`.mp4`）、音频（`.mp3`）、可执行文件（`.exe`）等。

第 9 章 第 9 章异常处理

知识要点

- 异常概念：程序运行过程中出现的错误，Python 使用异常对象来表示
- 常见异常类型：
 - `ZeroDivisionError`：除以零
 - `TypeError`：类型不匹配（如 `'a'+1`）
 - `ValueError`：值不合法（如 `int('abc')`）
 - `NameError`：变量未定义
 - `IndexError`：索引越界
 - `KeyError`：字典键不存在
 - `FileNotFoundError`：文件不存在
 - `SyntaxError`：语法错误
 - `AttributeError`：对象无此属性/方法
- 异常处理结构：
 - `try-except`：捕获异常
 - `try-except-else`：无异常时执行 `else`
 - `try-except-finally`：无论有无异常都执行 `finally`
 - `try-except-else-finally`：完整结构
- `raise` 语句：手动抛出异常
- 自定义异常：继承 `Exception` 类

异常处理错题精选

错题 1

以下代码的输出是什么？

```
try:
    x = int("abc")
except ValueError:
    print("A", end=' ')
finally:
    print("B", end=' ')
print("C")
```

A. A C B. A B C C. B C D. A B

参考答案与解析

答案： B

解析： `int('abc')` 抛出 `ValueError`，被 `except` 捕获 → 打印“A”。`finally` 始终执行 → 打印“B”。异常已被处理，程序继续执行 → 打印“C”。

错题 2

以下代码的输出是什么？

```
try:
    print("A")
    raise ValueError("出错了")
    print("B")
except ValueError as e:
    print("C")
else:
    print("D")
finally:
    print("E")
```

A. A C D E B. A C E C. A C D D. A B C E

参考答案与解析

答案： B

解析： `print('A')` → 输出 A； `raise` 抛出异常； `except` 捕获 → 输出 C； `else` 在无异常时执行，此处不执行； `finally` 始终执行 → 输出 E。

错题 3

以下代码的输出是什么？

```
def func():  
    try:  
        return 1  
    except:  
        return 2  
    finally:  
        return 3  
  
print(func())
```

A. 1 B. 2 C. 3 D. None

参考答案与解析

答案： C

解析： `finally` 块中的 `return` 会覆盖 `try` 块中的 `return`。虽然 `try` 准备返回 1，但在返回前 `finally` 强制执行，其中的 `return 3` 覆盖了之前的返回值。永远不要在 `finally` 里写 `return`！

第二部分

第 6 章面向对象程序设计详解

第 10 章 类的定义与使用

定义类的语法

```
class 类名(基类名):    # 基类可省略，默认为object
    """类的文档字符串"""
    def __init__(self, 参数): # 构造方法
        self.属性 = 参数    # 实例属性

    def 方法名(self, 参数):    # 实例方法
        pass
```

★ 要点

- 类名首字母一般大写（约定，非强制）
- 用 `class 类名: 定义`；若派生自其他类则把基类放括号中
- 实例化：`obj = 类名 (参数)`
- 访问成员：`对象名. 成员名`
- 用 `isinstance(obj, 类)` 测试对象是否为某类的实例
- 用 `type(obj)` 查看对象类型
- 类注释写在三引号中，通过 `类名.__doc__` 查看

示例

```
class Car(object):
    """汽车类"""
    def info(self):
        print('This is a car')

c = Car()    # 实例化
c.info()    # 调用方法
print(isinstance(c, Car)) # True
```

第 11 章 数据成员与成员方法

私有成员与公有成员（必考）

▲ 下划线命名规则

写法	类别	含义
<code>_xxx</code>	保护成员	单下划线开头，外部不建议直接访问
<code>__xxx</code>	私有成员	双下划线开头不包围，外部不能直接访问
<code>__xxx__</code>	特殊方法	双下划线包围，预定义，如 <code>__init__</code>

私有成员核心机制：

- 私有成员通过名称修饰（Name Mangling）变成 `_ 类名 __xxx`
- 外部可通过 `obj._ 类名 __xxx` 间接访问（但不推荐，破坏封装）
- `_xxx` 不能用 `from module import *` 导入

```
class A:
    def __init__(self):
        self._protected = 1      # 保护成员
        self.__private = 2      # 私有成员

a = A()
print(a._protected)             # 1（可访问但不建议）
# print(a.__private)           # AttributeError!
print(a._A__private)            # 2（名称修饰后可以）
```

▲ 易混淆点

- `__init__`——双下划线包围 → 系统特殊方法，不是私有！
- `__value`——双下划线不包围 → 私有成员
- `_value`——单下划线 → 保护成员（约定，非强制）

数据成员：实例属性 vs 类属性

★ 对比

类型	定义位置	特点
实例属性	<code>__init__</code> 中以 <code>self.xxx</code> 定义	每个对象独立，互不影响
类属性	类内、方法外直接定义	所有对象共享

```
class Demo:
    total = 0          # 类属性（共享）
    def __init__(self):
        self.value = 3 # 实例属性（独立）
        Demo.total += 1 # 类属性通过类名访问
```

经典应用：利用类属性控制最大对象数量。

```
class Demo:
    total = 0
    def __new__(cls, *args, **kwargs):
        if cls.total >= 3:
            raise Exception('最多只能创建3个对象')
        return object.__new__(cls)
    def __init__(self):
        Demo.total += 1
```

成员方法、类方法、静态方法、抽象方法

▲ 四种方法对比（重点）

类型	修饰符	首参数	访问实例	调用方式
实例方法	无	<code>self</code>	✓	对象名. 方法 ()
类方法	<code>@classmethod</code>	<code>cls</code>	×	类/对象名. 方法 ()
静态方法	<code>@staticmethod</code>	无要求	×	类/对象名. 方法 ()
抽象方法	<code>@abc.abstractmethod</code>	<code>self</code>	✓	派生类必须重写

核心要点：

- 函数 vs 方法：方法绑定类或对象，通过对象调用时 `self` 自动传递；普通函数不具备这个特点
- 所有实例方法必须至少有一个名为 `self` 的参数，且必须是第一个参数

- 静态方法和类方法不能直接访问实例成员，只能访问类成员
- 静态方法和类方法不属于任何实例，比实例方法节省开销

```
class Root:
    __total = 0

    def __init__(self, v):
        self.__value = v
        Root.__total += 1

    def show(self):                # 实例方法
        print('value:', self.__value)

    @classmethod
    def classShowTotal(cls):       # 类方法
        print('total:', cls.__total)

    @staticmethod
    def staticShowTotal():         # 静态方法
        print('total:', Root.__total)

r = Root(3)
r.show()                        # 实例方法
Root.classShowTotal()          # 类方法
Root.staticShowTotal()         # 静态方法
Root.show(r)                   # 通过类名调实例方法，需显式传self
```

属性（Property）

```
class Test:
    def __init__(self, value):
        self.__value = value

    @property
    def value(self):              # 只读属性
        return self.__value

# 或用 property() 函数定义可读可写属性：
def __get(self):
    return self.__value
```

```
def __set(self, v):
    self.__value = v
value = property(__get, __set)
```

动态性与混入机制（Mixin）

```
import types

class Car:
    price = 180000

car1 = Car()
Car.name = 'QQ' # 动态增加类属性
car1.color = 'Red' # 动态增加实例属性

def setSpeed(self, s):
    self.speed = s
car1.setSpeed = types.MethodType(setSpeed, car1) # 动态绑定方法
car1.setSpeed(50)
print(car1.speed) # 50
```

混入（Mixin）：根据需要动态为对象增加行为，适用于权限管理、角色管理等场景。

第 12 章 继承与多态

继承

继承规则：

- 派生类继承父类的公有成员，不能继承私有成员
- 调用基类方法：用 `super()` 或 基类名. 方法名 ()
- Python 支持多继承，方法搜索从左到右

```
class Person:
    def __init__(self, name='', age=20, sex='man'):
        self.__name = name
        self.__age = age
        self.__sex = sex

    def show(self):
        print(self.__name, self.__age, self.__sex)

class Teacher(Person):
    # 继承Person
    def __init__(self, name='', age=30, sex='man',
                  department='Computer'):
        super().__init__(name, age, sex) # 调用基类构造方法
        self.__department = department

    def show(self):
        # 重写（覆盖）父类方法
        super().show() # 调用父类show()
        print(self.__department)

lisi = Teacher('Li Si', 32, 'man', 'Math')
lisi.show()
```

▲ 继承中的构造方法

- 若派生类没有定义 `__init__`，会自动继承基类的构造方法
- 若派生类有定义 `__init__`，则不会自动调用基类的构造方法
- 必须用 `super().__init__()` 手动调用

多态

```
class Animal:
    def show(self):
        print('I am an animal.')

class Cat(Animal):
    def show(self):
        print('I am a cat.')

class Dog(Animal):
    def show(self):
        print('I am a dog.')

class Test(Animal):
    pass

for item in [Animal(), Cat(), Dog(), Test()]:
    item.show()
```

运行结果：

```
I am an animal.
I am a cat.
I am a dog.
I am an animal.
```


第 13 章 特殊方法与运算符重载

★ 常见特殊方法速查表（必背）

特殊方法	运算符/函数	说明
<code>__init__()</code>	构造方法	创建对象时自动调用
<code>__add__()</code>	<code>+</code>	加法
<code>__sub__()</code>	<code>-</code>	减法
<code>__mul__()</code>	<code>*</code>	乘法
<code>__truediv__()</code>	<code>/</code>	真除法
<code>__floordiv__()</code>	<code>//</code>	整除
<code>__mod__()</code>	<code>%</code>	取余
<code>__pow__()</code>	<code>**</code>	幂运算
<code>__eq__()</code>	<code>==</code>	判断值相等
<code>__lt__()</code>	<code><</code>	小于
<code>__gt__()</code>	<code>></code>	大于
<code>__len__()</code>	<code>len()</code>	长度
<code>__str__()</code>	<code>str()/print()</code>	字符串表示
<code>__getitem__()</code>	<code>obj[key]</code>	索引取值
<code>__setitem__()</code>	<code>obj[key]=v</code>	索引赋值
<code>__contains__()</code>	<code>in</code>	成员测试
<code>__call__()</code>	<code>obj()</code>	让对象可调用
<code>__bool__()</code>	<code>bool()</code>	真值判断
<code>__del__()</code>	析构方法	删除对象时自动调用

```
class Demo:
    def __init__(self, value):
        self.__value = value

    def __add__(self, other):
        if isinstance(other, (int, float)):
            return self.__value + other
        return None

    def __str__(self):
        return f'Demo({self.__value})'
```

```
def __call__(self, a, b):  
    return self.__value * a + b  
  
t = Demo(5)  
print(t + 3)      # 8  
print(t)          # Demo(5)  
print(t(2, 5))    # 15
```

第 14 章 本章重点案例赏析

自定义队列类

```
class MyDeque:
    def __init__(self, iterable=None, maxlen=10):
        if iterable is None:
            self._content = []
            self._current = 0
        else:
            self._content = list(iterable)
            self._current = len(iterable)
        self._size = maxlen

    def appendRight(self, v):
        if self._current < self._size:
            self._content.append(v)
            self._current += 1

    def popLeft(self):
        if self._content:
            self._current -= 1
            return self._content.pop(0)

    def rotate(self, k):
        self._content = self._content[-k:] + self._content[:-k]

    def reverse(self):
        self._content = self._content[::-1]

    def __len__(self):
        return self._current

    def __str__(self):
        return f'MyDeque({self._content}, maxlen={self._size})'
```

自定义栈类

```
class Stack:
    def __init__(self, maxlen=10):
        self._content = []
        self._current = 0
        self._size = maxlen

    def push(self, v):
        if self._current < self._size:
            self._content.append(v)
            self._current += 1

    def pop(self):
        if self._content:
            self._current -= 1
            return self._content.pop()

    def isEmpty(self):
        return not self._content

    def isFull(self):
        return self._current == self._size

    def __str__(self):
        return f'Stack({self._content}, maxlen={self._size})'
```

第 15 章 考试要点浓缩

▲ 必背速记

- 封装/继承/多态——面向对象三要素
- 下划线命名规则——`_x` 保护 / `__x` 私有 / `__x__` 特殊方法
- 私有成员名称修饰——`__x` $\rightarrow$ `_ 类名 __x`
- 实例属性 vs 类属性——实例独立 / 类共享
- 实例方法 vs 类方法 vs 静态方法
- 函数 vs 方法——方法绑定类/对象，`self` 自动传递
- `super()` 调用父类
- 派生类构造规则
- 特殊方法对应运算符
- 多态——同一方法在不同子类中不同表现

第 16 章 本章作业题

作业 6.1

面向对象程序设计的三要素分别为____、____ 和____。

参考答案与解析

答案依次为封装、继承、多态。

作业 6.3

假设 a 为类 A 的对象且包含私有数据成员 `__value`，在类的外部通过对象 a 将其私有数据成员 `__value` 的值设置为 3 的语句可以写作____。

参考答案与解析

答案：`a._A__value = 3`。Python 对以双下画线开路（不以双下画线结尾）的成员进行名称修饰（name mangling），`__value` 在类 A 内部被重命名为 `_A__value`。

作业 6.4

在 Python 中，不论类的名字是什么，构造方法都是____。

参考答案与解析

答案：`__init__`。

作业 6.23

判断对错：在 Python 中定义类时，如果某个成员名称前有两个下画线则表示是私有成员。

参考答案与解析

答案：✓ 正确。

作业 6.30

判断对错：在面向对象程序设计中，函数和方法是完全一样的，都必须为所有参数进行传值。

参考答案与解析

答案：× 错误。

作业 6.31

简单解释 Python 中以下画线开头和结束的变量名的含义。

参考答案与解析

1. `_xxx`: 单下画线开头, 保护成员。
2. `__xxx`: 双下画线开头, 私有成员, 名称修饰。
3. `__xxx__`: 双下画线开头和结尾, 特殊方法。

作业 6.32

继承 Person 类创建 Student 类, 添加设置学生专业的方法, 生成对象并显示信息。

参考答案与解析

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def info(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, major=""):
        super().__init__(name, age)
        self.major = major
    def set_major(self, major):
        self.major = major
    def info(self):
        super().info()
        print(f"Major: {self.major}")

stu = Student("Zhang San", 20)
stu.set_major("Computer Science")
stu.info()
```

第三部分

考前速背手册

速记卡片

一、运算符优先级表（核心必背）

口诀：幂 → 乘除 → 加减 → 比较 → *not* → *and* → *or*

优先级	运算符	名称	说明/示例
1（最高）	()	圆括号	改变优先级
2	**	幂运算	2**3=8（右结合）
3	* / // %	乘/除/整除/取余	同级左结合
4	+ -	加减	+ 还可连接
5	== != > < >= <=	比较	支持连用
5	is / is not	同一性	判引用是否同一
5	in / not in	成员测试	元素是否在序列中
6	not	逻辑非	取反
7	and	逻辑与	短路，返回末值
8（最低）	or	逻辑或	短路，返回末值

优先级经典示例

- $3 + 5 * 2 \rightarrow 3 + 10 = 13$ （先乘后加）
- $3 \text{ or } 5 \text{ and } 0 \rightarrow 3 \text{ or } 0 = 3$ （*and* 优先 *or*）
- $\text{not } 3 > 5 \rightarrow \text{not False} = \text{True}$

算术踩坑

- $4/2 = 2.0$ （不是 2）
- $-5//2 = -3$ （不是-2）
- $2**3**2 = 512$ （右结合）

and/or 返回值速记

- $3 \text{ and } 5 \rightarrow 5$ （左真，继续右，返回右）
- $0 \text{ and } 5 \rightarrow 0$ （左假，短路，返回左）
- $3 \text{ or } 5 \rightarrow 3$ （左真，短路，返回左）
- $0 \text{ or } 5 \rightarrow 5$ （左假，继续右，返回右）

注意：*and/or* 不返回 *True/False*，返回最后一个被计算表达式的值。

速记卡片

二、常用内置函数背诵表

函数	作用	注意点
<code>int()</code>	转整数	<code>int('ff', 16)</code> 指定进制
<code>float()</code>	转浮点	
<code>str()</code>	转字符串	任何对象都有
<code>bool()</code>	转布尔	0/空/None→False
<code>list()</code>	转列表	参数必须可迭代
<code>tuple()</code>	转元组	参数必须可迭代
<code>dict()</code>	转字典	参数为键值对序列
<code>set()</code>	转集合	自动去重
<code>len()</code>	求长度	支持所有序列
<code>max()</code>	求最大值	字符串按字典序
<code>min()</code>	求最小值	字符串按字典序
<code>sum()</code>	求和	元素必须为数值
<code>range()</code>	整数序列	左闭右开
<code>enumerate()</code>	返回 (索引, 值)	常与 for 搭配
<code>zip()</code>	打包多序列	按最短对齐
<code>map()</code>	函数作用于元素	返回迭代器
<code>sorted()</code>	排序返回新列表	不改变原序列
<code>reversed()</code>	反向迭代	返回迭代器
<code>eval()</code>	执行表达式字符串	有安全风险
<code>input()</code>	键盘输入	返回字符串
<code>print()</code>	输出	sep/end 参数
<code>type()</code>	查看类型	
<code>isinstance()</code>	类型判断	比 <code>type</code> 更推荐

高频踩坑

- `input()` 返回的是字符串
- `zip(a, b)` 按最短对齐
- `range(5, 0)` 空
- `sorted()` 返回新列表; `sort()` 原地排序, 返回 `None`

三种”导入”方式

- `import math → math.sqrt(9)`
- `from math import sqrt → sqrt(9)`
- `from math import * → 不推荐`

等价于 `False` 的值: `False`、`0`、`0.0`、`''`、`[]`、`()`、`{}`、`set()`、`None`

速记卡片

三、序列结构速查

五种序列对比	类型	定界	有序	可变	索引	空值
	列表	[]	✓	✓	✓	[]
	元组	()	✓	×	✓	()
	字典	{}	×	✓	键作下标	{}
	集合	{}	×	✓	×	set()
	字符串	'/'	✓	×	✓	''

列表核心方法：`append(x)` 尾部追加、`extend(seq)` 扩展、`insert(i,x)` 插入、`pop()` 弹出、`remove(x)` 按值删除、`del lst[i]` 按索引删、`copy()` 浅复制、`sort()` 原地排序（返回 None）、`reverse()` 原地反转（返回 None）

切片操作：`seq[start:stop:step]`（左闭右开）

- `a[:]` 完整复制（浅复制）
- `a[::2]` 步长 2（偶数下标）
- `a[::-1]` 逆序
- `a[0::-1]` 仅 [0] 一个元素（易错）

速记卡片

四、程序控制结构速记

for-else / while-else

- **else 执行：**循环正常结束（未被 **break** 打断）
- **else 不执行：**循环被 **break** 提前终止
- 典型应用：素数判定

break vs continue

- **break：**彻底退出当前循环，**else 不执行**
- **continue：**跳过本次循环剩余，**else 仍可能执行**

短路求值关键： $3 > 5$ and $a > b$ （ a 未定义） $\rightarrow$ 不抛异常

条件表达式真值规则

- **True 等价值：** $3+5$ 、 $[3]$ 、 $'a'$ 、 $(0,)$
- **False 等价值：** 0 、 $[]$ 、 $''$ 、**None**

速记卡片

五、函数核心速记

实参与可变序列（核心）	实参类型	函数内操作	影响外部
	不可变（int,str,tuple）	para= 新值（重赋值）	× 不影响
	可变（list,dict,set）	para[0]= 新值（改元素）	✓ 影响
	可变（list）	para=[新列表]（重赋值）	× 不影响

参数类型：位置参数、默认值参数、关键参数、`*args`（收集为元组）、`**kwargs`（收集为字典）

变量作用域判定

- 只引用不赋值 → 隐式全局变量
- 有赋值操作 → 隐式局部变量（除非 `global`）

经典踩坑（`UnboundLocalError`）

```
x = 3
def f():
    print(x) # 报错！
    x = 5
```

默认值陷阱

```
def f(lst=[]):
    lst.append(3); return lst
f() # [3]
f() # [3, 3] ← 累积！
```

速记卡片

六、面向对象与下划线命名（核心）

封装/继承/多态：封装（隐藏细节）、继承（复用代码）、多态（同方法不同表现）

下划线命名规则（必掌握）

写法	类别	含义
<code>__xx__</code>	系统特殊方法	预定义，如 <code>__init__</code>
<code>_xx</code>	保护成员	单下画线开头，外部仍可访问
<code>__xx</code>	私有成员	触发名称修饰 → <code>_类名__xx</code>
<code>_</code>	占位符	无关紧要的变量

易混淆点

- `__init__` → 特殊方法，不是私有！
- `__value` → 私有成员
- `_value` → 保护成员（约定）

速记卡片

七、字符串与文件操作速记

字符串方法：`find/index` 查找、`count` 统计、`split` 分割、`join` 连接（分隔符在前面）、`strip` 去空白、`lower/upper` 大小写、`replace` 替换

字符串不可变：所有“修改”方法返回新字符串，原字符串不变。

文件操作：`open()` → `'r'` 只读/`'w'` 只写（清空）/`'a'` 追加

with 语句模板

```
with open('f.txt', 'r', encoding='utf-8') as f:
    content = f.read()
# 自动关闭
```

速记卡片

八、异常处理与其他速记

常见异常: `ZeroDivisionError` 除以零、`TypeError` 类型不匹配、`ValueError` 值不合法、`NameError` 变量未定义、`IndexError` 索引越界、`KeyError` 键不存在

try-except-else-finally

- **try:** 监控代码块
- **except:** 有异常时执行
- **else:** 无异常时执行
- **finally:** 无论如何都执行

== / is / in

- **==:** 比较值
- **is:** 比较引用
- **in:** 元素在容器内

列表推导式 vs 生成器表达式

- 列表推导式 `[]` → 返回列表
- 生成器表达式 `()` → 返回生成器对象（惰性求值）

第四部分

模拟试卷

第 17 章 模拟试卷一

Python 程序设计基础模拟试卷（一）

考试时间：90 分钟 满分：100 分

选择题（每题 2 分，共 40 分）

一、选择题

1. 以下不属于 Python 语言特点的是（ ）。
A. 跨平台 B. 编译执行 C. 免费 D. 开源
2. 当一个 Python 模块被直接运行时，其 `__name__` 属性的值为（ ）。
A. 'main' B. '__main__' C. 模块文件名 D. None
3. 表达式 `0 and 5 or 3` 的值为（ ）。
A. 0 B. 5 C. 3 D. True
4. 表达式 `max(['111', '22'])` 的值为（ ）。
A. '111' B. '22' C. 111 D. 22
5. 表达式 `-7 // 3` 的值为（ ）。
A. -2 B. -3 C. 2 D. -2.33
6. 在 Python 中，判断两个变量是否指向同一内存地址应使用（ ）。
A. `==` B. `=` C. `is` D. `in`
7. 已知 `aList = [2, 4, 6, 8, 10, 12, 14]`，则切片 `aList[1:5:2]` 的值为（ ）。
A. [4, 8, 12] B. [4, 8] C. [2, 6, 10] D. [4, 6, 8, 10]
8. 字典和集合的界定符都是（ ）。
A. `[]` B. `()` C. `{}` D. `<>`

9. 已知 `vec = [[1, 2, 3], [4, 5, 6]]`, 则表达式 `[col for row in vec for col in row]` 的值为 ()。
- A. `[[1, 2, 3], [4, 5, 6]]` B. `[(1, 4), (2, 5), (3, 6)]` C. `[1, 2, 3, 4, 5, 6]` D. `[[1, 4], [2, 5], [3, 6]]`
10. 使用字典对象的 () 方法可以安全地获取指定键的值。
- A. `get()` B. `keys()` C. `items()` D. `pop()`
11. 在 `for-else` 循环结构中, `else` 块执行的条件是 ()。
- A. 循环被 `break` 打断时 B. 循环正常结束时 C. 循环条件第一次为假时 D. 无论何种情况
12. 作为条件表达式时等价于 `False` 的是 ()。
- A. `[0]` B. `'False'` C. `0` D. `(0,)`
13. 已知函数定义 `def func(a, b, *args): pass`, 通过 `func(1, 2, 3, 4, 5)` 调用, 则 `args` 的值为 ()。
- A. `[3, 4, 5]` B. `(3, 4, 5)` C. `3` D. `5`
14. 如果函数中没有 `return` 语句, 那么该函数的返回值为 ()。
- A. `0` B. `False` C. `None` D. 抛出异常
15. 以下代码的运行结果为 ()。
- ```
x = 10
def func():
 x = 20
func()
print(x)
```
- A. `10` B. `20` C. `None` D. 抛出异常
16. 面向对象程序设计的三大核心特征是 ( )。
- A. 类、对象、方法 B. 封装、继承、多态 C. 数据、方法、接口 D. 公有、私有、保护
17. 在 Python 中, 类的构造方法名称固定为 ( )。
- A. `init` B. `__init__` C. `constructor` D. `__new__`
18. 表达式 `len(''.join(['a', 'b', 'c', 'd']))` 的值为 ( )。
- A. `4` B. `7` C. `3` D. `8`

19. 使用上下文管理关键字 ( ) 可以自动管理文件对象。

- A. try                      B. with                      C. finally                      D. open

20. 执行 `int('hello')` 会触发 ( ) 异常。

- A. TypeError                      B. NameError                      C. ValueError                      D. SyntaxError

### 判断题（每题 2 分，共 20 分）

1. 在 Python 中，`3 > 5 and a > b` (a 和 b 均未定义) 会抛出 `NameError` 异常。
2. 字符串属于可变序列，可以通过下标修改其中的字符。
3. 元组的元素不能被修改，因此也不能用 `del` 删除整个元组。
4. 调用带有默认值参数的函数时，不能为其传递任何值。
5. `sort()` 和 `sorted()` 功能完全相同。
6. 在 Python 中定义类时，以双下画线开头且不以双下画线结尾的成员表示私有成员。
7. `find()` 和 `index()` 方法功能完全相同。
8. 使用 `open()` 以 'w' 模式打开已存在的文件时，原有内容会被清空。
9. Python 属于强类型编程语言。
10. 生成器表达式使用方括号 `[]` 定义，返回一个列表。

### 简答题（每题 10 分，共 20 分）

1. (10 分) 简述 Python 中局部变量和全局变量的判定规则，并说明在函数内部如何修改全局变量的值。
2. (10 分) 简述 Python 中以下画线开头和结束的变量名的含义及区别。

### 编程题（每题 15 分，共 30 分）

1. (15 分) 编写函数 `is_palindrome(s)`，判断字符串是否为回文。
2. (15 分) 编写类 `BankAccount`，包含构造方法、存款、取款、显示信息功能。



## 参考答案

### 参考答案与解析

#### 一、选择题

1. B
2. B
3. C
4. B
5. B
6. C
7. B
8. C
9. C
10. A
11. B
12. C
13. B
14. C
15. A
16. B
17. B
18. B
19. B
20. C

#### 二、判断题

1. ×（短路求值，不会抛出异常）
2. ×（字符串不可变）
3. ×（可以用 del 删除整个元组）
4. ×（可以为默认值参数传递新值）
5. ×（sort 原地排序返回 None，sorted 返回新列表）

## 第 18 章 模拟试卷二

### Python 程序设计基础模拟试卷（二）

考试时间：90 分钟      满分：100 分

#### 选择题（每题 2 分，共 40 分）

##### 一、选择题

1. 最不推荐的导入方式是（ ）。

A. `import math` B. `from math import sqrt` C. `from math import *` D. `import math as m`

2. 常用于实现模块自测的代码结构是（ ）。

A. `if __name__ == '__main__':` B. `if __main__ == '__name__':` C. `while __name__ == '__main__':` D. `def __main__():`

3. 合法的 Python 变量名是（ ）。

A. `2name` B. `for` C. `_name` D. `name@`

4. 表达式 `2 * 3 ** 2` 的值为（ ）。

A. 36 B. 18 C. 12 D. 64

5. 表达式 `eval('3+4*2')` 的值为（ ）。

A. 14 B. 11 C. 24 D. `'3+4*2'`

6. 表达式 `'abc' > 'ABC'` 的值为（ ）。

A. `True` B. `False` C. `'abc'` D. 抛出异常

7. 已知 `s = 'Python'`，则 `s[::-1]` 的值为（ ）。

A. `'nohtyP'` B. `'P'` C. `'Python'` D. `'n'`

8. 列表删除首次出现的指定元素的方法是（ ）。

A. `pop()` B. `remove()` C. `delete()` D. `discard()`

9. 执行 `a, *b = [1, 2, 3, 4]` 后, `b` 的值为 ( )。
- A. `[1]`                      B. `[2, 3, 4]`                      C. `[4]`                      D. `[2]`
10. 字典统计元素出现次数时常用 ( ) 获取不存在的键的默认值。
- A. `d[key]`                      B. `d.get(key, 0)`                      C. `d.keys()`                      D. `d.pop(key)`
11. `continue` 语句的作用是 ( )。
- A. 提前结束整个循环 B. 结束本次循环进入下一次迭代 C. 终止程序运行 D. 跳过循环条件
12. 以下代码的运行结果为 ( )。
- ```
for i in range(5):  
    if i == 3: break  
    print(i, end='')
```
- A. 01234 B. 012 C. 0123 D. 01234
13. `def func(**kwargs): pass`, 调用 `func(x=1,y=2)`, `kwargs` 的值为 ()。
- A. `{'x':1, 'y':2}` B. `(x=1,y=2)` C. `[1,2]` D. `(1,2)`
14. 连续调用三次 `func(lst=[])`, 第三次返回 ()。
- A. `[1]` B. `[1,1]` C. `[1,1,1]` D. `[]`
15. `f = lambda x: x*x+1`, `f(3)` 的值为 ()。
- A. 7 B. 9 C. 10 D. 6
16. 子类调用父类构造方法应使用 ()。
- A. `parent.__init__(self)` B. `super().__init__()` C. `self.parent.__init__()` D. `base.__init__()`
17. 类 A 中定义了 `__value`, 外部正确访问是 ()。
- A. `a.__value` B. `a._value` C. `a._A__value` D. `A.__value`
18. `'hello world'.split()[1]` 的值为 ()。
- A. `'hello'` B. `'world'` C. `'hello world'` D. `''`
19. 以 `'a'` 模式打开文件, 文件指针位于 ()。
- A. 文件开头 B. 文件尾部 C. 文件中间 D. 不确定
20. `finally` 子句的执行时机是 ()。
- A. 有异常时 B. 无异常时 C. 无论有无异常都执行 D. 仅异常未被捕获时

判断题（每题 2 分，共 20 分）

1. Python 变量不需要事先声明类型，这是动态类型语言的特点。
2. 列表和元组的主要区别是列表用方括号，元组用圆括号。
3. `del` 命令可以删除列表元素，也可以删除整个变量。
4. `from math import *` 是推荐的最佳实践。
5. Python 中 `/` 是真除法，结果始终为浮点数。
6. 在派生类中不能直接访问基类的私有成员。
7. 集合支持使用下标索引访问元素。
8. 字符串的 `strip()` 方法会修改原字符串。
9. 以 `'r'` 模式打开不存在的文件时，会自动创建该文件。
10. 在 `for-else` 中，循环被 `break` 打断，`else` 块仍会执行。

简答题（每题 10 分，共 20 分）

1. (10 分) 举例说明函数内修改形参影响外部实参的情况。
2. (10 分) 简述 `with` 语句的作用和优势，写出读取文件的典型代码。

编程题（每题 15 分，共 30 分）

1. (15 分) 生成 20 个随机整数（1~100），删除所有能被 3 整除的数。
2. (15 分) 编写函数 `count_elements(lst)`，用字典统计元素出现次数。

参考答案

参考答案与解析

一、选择题

1. C
2. A
3. C
4. B
5. B
6. A
7. A
8. B
9. B
10. B
11. B
12. B
13. A
14. C
15. C
16. B
17. C
18. B
19. B
20. C

二、判断题

1. ✓
2. ×（本质区别是可变性）
3. ✓
4. ×（污染命名空间）
5. ✓

第 A 章 错题集精选

变量引用与对象绑定

错题 1

以下代码的输出是什么？

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

- A. [1, 2, 3] B. [1, 2, 3, 4] C. 报错 D. [1, 2, 3, [4]]

参考答案与解析

答案：B 解析：b = a 让 b 和 a 指向同一个列表，b.append(4) 修改的是该列表本身。

错题 2

以下代码的输出是什么？

```
a = [1, 2, 3]
b = a[:]
b.append(4)
print(a)
```

- A. [1, 2, 3, 4] B. [1, 2, 3] C. [4] D. 报错

参考答案与解析

答案：B 解析：b = a[:] 是浅拷贝创建新列表，修改 b 不影响 a。

错题 3

```
a = [1, 2, 3]
b = a
a = [4, 5, 6]
print(b)
```

- A. [1, 2, 3] B. [4, 5, 6] C. [1,2,3,4,5,6] D. 报错

参考答案与解析

答案：A 解析：a = [4,5,6] 让 a 重新绑定到新列表，b 仍指向旧列表。重新绑定不影响另一变量。

错题 4

```
x = [1, 2, 3]
y = x
x.append(4)
x = [5, 6]
y.append(7)
print(y)
```

- A. [1,2,3,4,7] B. [5,6,7] C. [1,2,3,4,5,6,7] D. [5,6]

参考答案与解析

答案：A

错题 5

```
x = [1, 2, 3]
y = x
x = x + [4, 5]
print(y)
```

- A. [1,2,3,4,5] B. [1,2,3] C. [4,5] D. 报错

参考答案与解析

答案：B 解析：x = x + [4,5] 创建新列表，x 重新绑定，y 仍指向旧列表。

默认值参数陷阱

错题 6

```
def func(a, b=[]):  
    b.append(a); return b  
print(func(1))  
print(func(2))
```

- A. [1] [2] B. [1] [1, 2] C. [1,2] [1,2] D. 报错

参考答案与解析

答案： B 解析： 默认参数 `b=[]` 只创建一次，多次调用累积。

错题 7

```
def func(a, b=[]):  
    b = b + [a]; return b  
print(func(1))  
print(func(2))
```

- A. [1] [2] B. [1] [1,2] C. [1] [2,1] D. 报错

参考答案与解析

答案： A 解析： `b = b + [a]` 创建新列表，不修改默认值本身。

变量作用域

错题 8

```
x = 10  
def func():  
    print(x)  
    x = 5  
func()
```

- A. 10 B. 5 C. None D. 报错

参考答案与解析

答案：D 解析：函数体内有赋值 `x=5`，整个函数的 `x` 都被视为局部变量，`print(x)` 时局部 `x` 未定义，报 `UnboundLocalError`。

错题 9

```
x = 1; y = 2
def func():
    global x; x = 10; y = 20
func(); print(x, y)
```

A. 10 2 B. 10 20 C. 1 20 D. 1 2

参考答案与解析

答案：A 解析：`global x` 声明修改了全局 `x`；`y = 20` 没有 `global` 声明，创建局部 `y`。

推导式与运算符

错题 10

以下哪个返回生成器？A. `[x**2 for x in range(5)]` B. `(x**2 for x in range(5))`
C. `{x**2 for x in range(5)}` D. `{x: x**2 for x in range(5)}`

参考答案与解析

答案：B 解析：() 是生成器表达式，返回生成器对象。

错题 11

```
print(2 ** 3 ** 2)
```

A. 64 B. 512 C. 36 D. 12

参考答案与解析

答案：B 解析：`**` 右结合，`2**(3**2)=2**9=512`。

错题 12

```
x = 5; y = 10
z = x > y and x or y
print(z)
```

A. 5 B. 10 C. True D. False

参考答案与解析

答案： B 解析： $x > y$ 为 False，短路返回 $y=10$ 。

字典与集合

错题 13

```
d = {}
d[True] = "yes"; d[1] = "no"
d[False] = "maybe"; d[0] = "never"
print(d)
```

A. 4 项 B. {True:'no', False:'never'} C. {1:'no',0:'never'} D. 报错

参考答案与解析

答案： B 解析： $\text{True}==1$ ， $\text{False}==0$ ，后面的覆盖前面的。

错题 14

```
d = {"name": "Alice", "age": 20}
print(d.get("gender"))
```

A. None B. 报错 C. KeyError D. False

参考答案与解析

答案： A 解析： `get()` 不存在返回 None，不报错。

循环控制

错题 15

```
for i in range(3): pass
print(i)
```

- A. 0 B. 2 C. 3 D. 报错

参考答案与解析

答案：B 解析：Python 中 for 循环变量泄露到外部，最后一次 $i=2$ 。

错题 16

```
for i in range(5):
    if i == 3: break
    print(i, end=' ')
else:
    print("正常结束")
```

- A. 0 1 2 正常结束 B. 0 1 2 C. 0 1 2 3 4 正常结束 D. 0 1 2 3

参考答案与解析

答案：B 解析：break 打断循环，else 不执行。

函数调用综合

错题 17

```
data = [1, 2, 3, 4, 5]
result = data[1:4]
print(result)
```

- A. [2, 3, 4] B. [1, 2, 3, 4] C. [2, 3, 4, 5] D. [1, 2, 3]

参考答案与解析

答案：A 解析：左闭右开，`data[1:4]` 取索引 1,2,3。

错题 18

```
def func(a, b, c=3, *args, **kwargs):  
    print(a, b, c, args, kwargs)  
func(1, 2, 4, 5, 6, x=10, y=20)
```

- A. 1 2 3 (5,6) {'x':10,'y':20} B. 1 2 3 (4,5,6) {'x':10,'y':20} C. 1 2 4 (5,6) {'x':10,'y':20}
D. 1 2 4 () {'x':10,'y':20}

参考答案与解析

答案： C